Pubpdf 使い方マニュアル

Markdown から本の体裁の PDF を作る

文書番号	PUBPDF-MAN-001
版数	1.7
発行日	2026-11-14
作成	Pubpdf プロジェクト

バージョン履歴

版	日付	変更内容
0.1	2026-11-06	初版
0.2	2026-11-06	VS Code 拡張機能、表の題、機密表示の位置、フォント、囲み、ファイル名つきコードブロックの説明を追加
0.3	2026-11-06	ページの通し番号とバージョン履歴の説明を追加
0.4	2026-11-06	文字の大きさ、余白、番号の形式、式の自動の番号、改ページ、図の寄せ、コードの行番号の説明を追加
0.5	2026-11-07	見本の原稿 test.md の案内を追加
0.6	2026-11-07	文字を右や中央に寄せる書き方の説明を追加
0.7	2026-11-07	参考文献、図と表の一覧、付録、概要、値の差し込みの説明を追加
0.8	2026-11-07	段組み、ヘッダとフッタの文字、透かし、ロゴと組織名、テーマの色、表の体裁、コードの配色、PDF の文書情報と規格の説明を追加
0.9	2026-11-07	Mermaid にない図の説明と、URL の画像を読み込まないことの説明を追加
1.0	2026-11-08	承認欄、履歴の列、見出しの番号の形、節ごとの図表の番号、表の網掛け、セルの中の改行、HTML の表の説明を追加
1.1	2026-11-08	スイムレーン、FTA、方眼の図、記入の指示の囲み、履歴の様式、ヘッダとフッタの罫線、表紙の枠、表の列の幅と文字の大きさの説明を追加
1.2	2026-11-08	ER 図と画面遷移図の説明、Mermaid の図の見た目の説明、見本とテンプレートの案内を追加
1.3	2026-11-09	H1 が 1 つでも題にしないことと、表紙がないときは title に書いた題だけを 1 ページ目に出すことを追記
1.4	2026-11-11	脚注を参考文献の一覧にする方法、大きなコードを 1 ページに収める設定、中間ファイルの置き場所の設定を追加
1.5	2026-11-11	表を中身の幅に合わせる指定(.fit、.full、table-fit)、コードの添付(code-attach)、コードの見せ方(code-display)、添付に切り替えたコードの文(code-attach-note)、コードのファイルの参照(include)を追加。本書の短い表を中身の幅にした
1.6	2026-11-12	文中のコードの背景(inline-code-background)、パスと URL の書き方、画像と図の幅の指定、差分(diff)の表示、HTML のタグ、Qiita と Zenn の記法(数式、画像の題、絵文字、囲みの題)、空行の抜けの警告、\ で始まる語と生の URL の扱いを追加。画像と図は既定で中央に置き、fig-align で寄せを変えられることを追記
1.7	2026-11-14	1 ページに入らないコードを付録へ移す(code-display の appendix、code-appendix-title、code-appendix-note)ことと、コードを既定では添付しないことを追記。表のセルの箇条書き、[^1,2] の脚注、図の .fit、文章の点検、ライブプレビュー(作る PDF と同じ表示、状態の表示、編集中の箇所の目印、ダブルクリックで原稿へ移る操作)、表の見やすい表示(列を揃える表示と、行の終わりの Enter で行を足すこと)、設定パネルの「編集中の表示」、Mermaid の図の再利用を追加

目次

本書について 7
目的別の読み方 7
本書自身について 7
第 1 章 Pubpdf とは 9
1.1 できること 9
1.2 基本の考え方 9
1.3 PDF ができるまで 10
1.4 作られるファイル 10
第 2 章 はじめての PDF 11
2.1 必要なもの 11
2.2 拡張機能を入れる 11
2.3 最初の文書を作る 11
2.4 設定を足す 12
2.5 見本を見る 12
2.6 出力先を変える 12
第 3 章 文書情報の書き方 13
3.1 frontmatter とは 13
3.2 書ける項目 13
3.3 バージョン履歴 14
3.4 承認欄 15
3.5 概要 16
3.6 タイトルを書かないとき 16
3.7 表紙がないとき 16
3.8 書き間違えたとき 17
3.9 frontmatter に書いても効かないもの 17
3.10 設定の優先順位 17
第 4 章 設定一覧 19
4.1 表紙 22
4.2 機密表示 22
4.3 透かし 23
4.4 目次 23
4.5 図と表の一覧 24
4.6 参考文献 24
4.7 用紙サイズと向き 24
4.8 余白 25
4.9 段組み 25
4.10 文字の大きさと行間 25
4.11 ページ番号 26
4.12 ヘッダとフッタの文字 26

4.13 見出しと図表の番号 27
4.14 式の番号 28
4.15 コードの行番号 28
4.16 コードの見せ方 29
4.17 コードの添付 30
4.18 章の前の改ページ 30
4.19 色と体裁 30
4.20 フォント 32
4.21 PDF の規格 33
第 5 章 本として組む 35
5.1 奇数ページ始まり 35
5.2 綴じ代 36
5.3 複数の md を 1 冊にする(合本) 36
第 6 章 原稿の書き方 39
6.1 段落と強調 39
6.2 脚注を参考文献の一覧にする 39
6.3 文字を右や中央に寄せる 40
6.4 段組み 40
6.5 文字の修飾 41
6.6 パスと URL 41
6.7 HTML のタグ 42
6.8 リスト 42
6.9 表 43
6.10 コード 46
6.11 数式 49
6.12 定理と証明 50
6.13 囲み(Qiita と Zenn の記法) 51
6.14 記入の指示の囲み 52
6.15 注意書きの囲み(callout) 52
6.16 Mermaid の図 52
6.17 Mermaid にない図 54
6.18 画像 66
6.19 改ページと横向きのページ 67
6.20 Typst を直接書く 68
6.21 文献の引用 68
6.22 文書の値を本文に差し込む 69
6.23 リンク 69
6.24 見出しへの参照 70
6.25 書くときの注意 70
第 7 章 VS Code での操作 71
7.1 コマンド 71
7.2 PDF を作る 71

7.3 設定パネル 72
7.4 プレビュー 73
7.5 ライブプレビュー 73
7.6 文章の点検 75
7.7 表の見やすい表示(試験的) 76
7.8 エラーと警告 77
7.9 設定のキー 77
第 8 章 困ったときは 81
8.1 エラーで止まるとき 81
8.2 警告が出るとき 82
8.3 仕上がりがおかしいとき 85
8.4 既知の制限 85
8.5 動作を確かめた Quarto の版 86

本書について

本書は、Markdown で書いた原稿を Pubpdf で PDF にする人のための手引きです。Quarto や Typst の知識がなくても読めるように書いています。

PDF は、VS Code 拡張機能で作ります。

目的別の読み方

表 1: 目的別の読み方

やりたいこと	読む章
まず 1 つ PDF を作りたい	はじめての PDF
タイトルや著者を入れたい	文書情報の書き方
設定の値を調べたい	設定一覧
両面印刷や製本を前提に組みたい、複数の md を 1 冊にしたい	本として組む
段組み、ヘッダとフッタの文字、透かし、色、PDF の規格を指定したい	設定一覧
表、コード、数式、囲み、段組みの書き方を知りたい	原稿の書き方
ディレクトリ構造、グラフ、工程表などの図を描きたい	原稿の書き方
VS Code のボタンや設定パネルの使い方を知りたい	VS Code での操作
エラーや警告が出た	困ったときは

本書自身について

本書も Pubpdf で組んでいます。原稿は`docs/manual/`にあり、`manual.md`を開いて PDF を作ると同じものができます。

右上と左下の Confidential は、機密表示の見本として付けたものです。本書が機密文書という意味ではありません。

第 1 章　Pubpdf とは

Pubpdf は、Markdown の原稿から PDF を作るツールです。組版には Quarto と Typst を使いますが、それらの設定ファイルを利用者が書く必要はありません。

1.1　できること

- 1 つの md から 1 つの PDF を作る
- 複数の md を 1 冊にまとめる
- 表紙、機密表示、目次、ページ番号を設定で切り替える
- 文字の大きさ、行間、余白、用紙の種類と向き、見出しと図表の番号の形式を設定で変える
- frontmatter に書いた履歴と概要を、それぞれのページにする。表紙に承認欄と押印欄、二重の枠を付ける
- 設計書の形の表を組む(セルの結合と網掛け、2 段の見出し、セルの中の改行、HTML の表)。短い表は中身の幅に合わせられる
- 見出しの番号を「1. → 1.1 → (1) → ①」のような形にし、図表の番号を節ごと(表 1.1-1)に振る
- 参考文献を引用し、文献の一覧、図と表の一覧、付録を組む。脚注を参考文献の一覧にまとめ、一覧を置く位置も決められる
- 本文を段組みにし、ヘッダとフッタに文字を入れ、透かしと表紙のロゴを付ける
- テーマの色、表の体裁、コードの配色を選ぶ
- PDF の文書情報を指定し、PDF/A と PDF/UA で出力する
- 章を奇数ページから始め、綴じ代を取る(両面印刷や製本の前提)
- 表、コード(ファイルの参照、大きさに合わせた縮小と添付ファイルへの切り替え)、数式、定理、Mermaid の図(ほかの図と同じ配色にそろえる)、注意書きの囲みを組む
- ディレクトリ構造、グラフ、工程表、ビット配置、層構造、波形、作業分解、画面の模式図、JSON と YAML、ネットワーク、ユースケース、スイムレーンの業務フロー、FTA、帳票と画面の方眼、ER、画面遷移の図を描く
- テンプレート用の記入の囲みを書き、仕上げるときにまとめて消す
- PDF にしおりを付ける(見出しから自動で作ります)
- フォントを指定し、OS に入れていないフォントも読み込む
- VS Code の中で作成、プレビュー、設定の変更をする

1.2　基本の考え方

原稿は Markdown だけ
: タイトルや著者などの情報は、原稿の先頭に YAML で書きます。設定ファイルを別に持ちません。

原稿は書き換えない
: 組版に必要な変換は中間コピーに対して行います。PDF を何度作っても、原稿は変わりません。設定パネルで変更したときだけ、frontmatter を書き換えます。

構成ファイルはツールが作る

`_quarto.yml` やテンプレートは、毎回ツールが作り直します。同じフォルダに自分の `_quarto.yml` があっても、影響を受けません。

1.3 PDF ができるまで

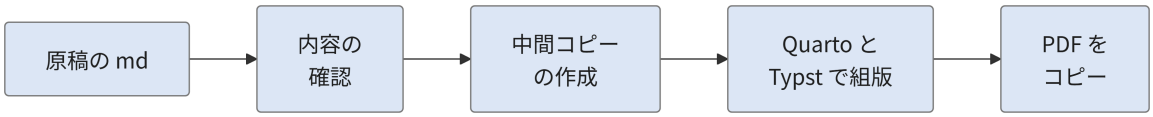


図 1-1: 原稿から PDF ができるまでの流れ

図 1-1 の各段階では、次のことを行います。

1. **内容の確認**: 設定の値を確かめ、章ファイルと画像があるかを確認します。誤りがあれば、ここで警告かエラーを出します。
2. **中間コピーの作成**: 原稿と画像を隠しフォルダに複製し、見出しへのリンクや Mermaid の記法を変換します。
3. **組版**: Quarto と Typst で PDF を作ります。
4. **コピー**: 完成した PDF を出力先へコピーします。

1.4 作られるファイル

表 1-1: Pubpdf が作るファイル

場所	中身	扱い
`<原稿名>.pdf`	完成した PDF	原稿と同じフォルダに作ります
`.pubpdf/build/`	中間コピー、`_quarto.yml`、テンプレート	毎回作り直すので、消しても問題ありません。既定ではワークスペースのフォルダの直下に作ります

中間ファイルの置き場所は、VS Code の設定 `pubpdf.buildDirectory` で選べます。`workspace` (既定)はワークスペースのフォルダの直下、`document` は原稿と同じフォルダ、`global` は拡張機能の保管場所で、プロジェクトのフォルダには作りません。ワークスペースの外の原稿は、原稿と同じフォルダに作ります。

Git で管理するときは、`.gitignore` に `.pubpdf/` の行を足しておきます。ワークスペースのフォルダが Git リポジトリなら、VS Code 拡張機能は最初に PDF を作ったときに追加するか尋ねます。

第 2 章　はじめての PDF

VS Code で最初の PDF を作るまでの手順です。

2.1　必要なもの

表 2-1: 必要なソフト

ソフト	用途	補足
Quarto 1.10.18 以上	組版	https://quarto.org/ から入手します
VS Code 1.90 以上	拡張機能	
日本語フォント	本文	Windows なら標準のフォントで動きます
Edge か Chrome	Mermaid の図	図を使わなければ不要です

2.2　拡張機能を入れる

最新の Release から `pubpdf-1.7.3.vsix` をダウンロードして、VS Code に入れます。ダウンロードしたフォルダのターミナルで次のコマンドを実行するか、拡張機能のビューの「…」メニューから「VSIX からのインストール」を選びます。

```
code --install-extension pubpdf-1.7.3.vsix
```

入れたら、コマンドパレット(`Ctrl+Shift+P`)で `Pubpdf: 環境を確認` を実行します。Quarto、フォント、ブラウザの状態が一覧で出ます(Quarto が見つからないときは Quarto の行だけです)。Quarto かフォントが足りないときは、項目を選ぶと入手先が開きます。ブラウザは Mermaid を使うときだけ必要です。

2.3　最初の文書を作る

次の内容を `sample.md` として保存します。文字コードは UTF-8 にしてください。先頭の `---` で挟んだ部分は frontmatter と呼び、文書の情報を書く場所です。

```
---
title: はじめての文書
author: 開発チーム
---

# 概要

Pubpdf で作った最初の文書です。

## 目的

本文を Markdown で書きます。
```

`sample.md` を開いたまま、エディタ右上の PDF のボタンを押します。ボタンは原稿にフォーカスがあるときに出ます。ボタンが出ないときは、ワークスペースを信頼しているか確認してく

ださい(制限モードでは拡張機能は動きません)。未保存の変更は保存してから組みます。完成すると通知が出て、原稿と同じフォルダに`sample.pdf`ができます。通知の「プレビュー」を押すと、VS Code の中で確認できます。

1 ページ目にタイトル、著者、目次が入り、2 ページ目から本文が始まります。

2.4 設定を足す

エディタ右上の歯車のボタンで設定パネルを開くと、表紙や機密表示を選べます。選んだ内容は frontmatter に書き込まれます。frontmatter に直接書いても同じです。

```
---
title: はじめての文書
author: 開発チーム
pubpdf:
  cover: simple
  confidential: 社外秘
---
```

もう一度 PDF を作ると(設定パネルの「PDF を作成」でも作れます)、1 ページ目が表紙になり、各ページの右上に「社外秘」が出ます。使える設定の全体は設定一覧にまとめています。

2.5 見本を見る

Pubpdf のフォルダにある`test.md`は、代表的な機能を使った見本の原稿です。表紙、バージョン履歴、概要、値の差し込み、表と図の番号、文献の引用と参考文献の一覧、数式の番号、ファイル名つきのコード、色付きの囲み、文字の寄せ、横向きのページ、改ページを使っています。PDF を作って、原稿と見比べてください。

`samples`フォルダには、IPA の設計書の様式に沿った記入例があります。要件定義書、基本設計書、システム要求仕様書、システム・アーキテクチャ設計書、ソフトウェア要求仕様書、ソフトウェア・アーキテクチャ設計書、ソフトウェア詳細設計書、安全要求仕様書、ソフトウェア総合テスト仕様書・報告書の 9 つです。`templates`フォルダには、同じ様式の空のテンプレートがあり、記入の指示を`::: {.guide}`の囲みに書いています。

2.6 出力先を変える

出力先は、`pubpdf.output`で指定できます。原稿のフォルダからの相対パスで、拡張子が`.pdf`のものだけを指定できます。受け取った原稿が他のファイルを上書きしないよう、原稿のフォルダの外は指定できません。外へ出すときは、VS Code の設定`pubpdf.outputDirectory`を使います。

```
pubpdf:
  output: dist/sample.pdf
```

第 3 章　文書情報の書き方

3.1　frontmatter とは

原稿の先頭で`---`に挟んだ YAML の部分を frontmatter と呼びます。文書の情報と設定は、すべてここに書きます。

```yaml
---
title: 製品仕様書
subtitle: 基本設計編
author: 開発チーム
date: 2026-11-06
version: "1.2"
doc-number: DOC-001
pubpdf:
  cover: formal
---
```

3.2　書ける項目

表 3-1: frontmatter の項目

キー	内容	表示される場所
`title`	文書名	表紙か 1 ページ目の先頭、PDF のプロパティ
`subtitle`	副題	表紙か 1 ページ目の先頭
`author`	著者。複数ならリスト	表紙か 1 ページ目の先頭、PDF のプロパティ
`organization`	組織名	表紙か 1 ページ目の先頭
`date`	発行日	表紙か 1 ページ目の先頭
`version`	版数	`formal`の表紙
`doc-number`	文書番号	`formal`の表紙
`abstract`	概要。Markdown で書けます	概要のページか、1 ページ目の題の下
`keywords`	キーワード。文字かリスト	PDF のプロパティ
`subject`	主題	PDF のプロパティ
`history`	版ごとの変更の記録	バージョン履歴のページ
`approval`	承認欄の役割、名前、日付	表紙か 1 ページ目の題の下
`lang`	言語。既定は`ja`	図や callout の名前(目次の題、「第 1 章」、表紙の項目名は日本語のまま)
`pubpdf`	ツールの設定	設定一覧を参照

`title`、`subtitle`、`version`、`doc-number`は Markdown として読みます。`author`、`organization`、`date`、`keywords`、`subject`は、書いたとおりの文字で出します。`date`に`today`と書くと、組んだ日の日付になります。

> **版数は引用符で囲む**
>
> `version: 1.10` と書くと YAML が数値として読み、`1.1` と表示されます。`version: "1.10"` のように引用符で囲んでください。

複数の著者は、次のように書きます。

```yaml
author:
  - 開発チーム
  - 品質チーム
```

`keywords` と `subject` は、PDF を開いたアプリの「プロパティ」に出る情報です。ページには出ません。`keywords` は、リストか、`,` や `、` で区切った文字で書きます。

```yaml
keywords: [仕様書, 製品 A]
subject: 製品 A の基本設計
```

3.3 バージョン履歴

`history` に履歴を書くと、表紙の次に「バージョン履歴」のページが入り、その次に目次が続きます。

```yaml
---
title: 製品仕様書
version: "1.1"
history:
  - version: 1.0
    date: 2026-10-01
    changes: 初版
  - version: 1.1
    date: 2026-11-06
    changes:
      - 第 2 章を追加
      - 誤字を修正
pubpdf:
  cover: formal
---
```

表 3-2: history の項目

キー	表の列	内容
`version`	版	版の番号
`date`	日付	変更した日
`changes`	変更内容	変更の内容。リストで書くと 1 項目ずつ改行します
`author`	作成者	変更した人
`approver`	承認者	変更を承認した人
`note`	備考	補足

- 表の列は、版、日付、変更内容、作成者、承認者、備考の順です。作成者、承認者、備考の列は、どれかの版に書いたときだけ出ます。行は書いた順に並びます。
- 設定の`history-number: true`で、表の左に 1 から数える項番の列を出します。
- 設定の`history-columns`で、列の順と見出しを決められます。列は`number`(項番)、`version`、`date`、`changes`、`author`、`approver`、`note`から選び、`version: バージョン`のように書くと見出しも変わります。指定した列は、値を書いていなくても出ます。
- 設定の`history-title`でページの題を、`history-blank-rows`で手書き用の空の行の数を変えられます。版を書いていなくても、空の行を指定すれば履歴のページを出します。

IPA の様式(項番、日付、バージョン、改訂内容、備考)にするときは、次のように書きます。

```
pubpdf:
  history-title: 改訂履歴
  history-columns: [number, date, "version: バージョン", "changes: 改訂内容",
note]
  history-blank-rows: 10
```

- 値は Markdown として読まず、書いたとおりの文字で出します。`history`の`version`は、引用符がなくても`1.10`のまま出ます。
- 表紙の版数は`version`から出します。`history`からは取りません。
- 履歴のページは目次に載りません。
- 表が 1 ページに収まらないときは、見出し行を繰り返して次のページへ続きます。
- 表紙がないときは、1 ページ目の題の下に表を出し、目次は次のページから始めます。
- 合本では親 md の`history`を使います。章の md に書いても無視し、警告を出します。

3.4 承認欄

`approval`に役割ごとの名前と日付を書くと、表紙に承認欄を出します。役割の順に、左から右へ並びます。

```
approval:
  - role: 承認
    name: 山田 太郎
    date: 2026-11-07
  - role: 審査
  - role: 作成
    name: 佐藤 一郎
    date: 2026-11-05
pubpdf:
  cover: formal
  approval-stamp: true
```

表 3-3: approval の項目

キー	内容
`role`	役割。必須です
`name`	名前。書かなければ空欄です
`date`	日付。書かなければ空欄です

- 役割は 8 個まで書けます。
- 設定の `approval-stamp: true` で、役割の下に押印や署名のための空欄を足します。名前も日付も書かなければ、指定しなくても空欄を出します。
- `formal` の表紙では文書管理情報の表の上に、`simple` の表紙では著者の下に出します。表紙がないときは、1 ページ目の題の下に出します。
- 値は Markdown として読まず、書いたとおりの文字で出します。

3.5 概要

`abstract` に書いた内容を、「概要」の見出しを付けて出します。段落を分けるときは `|` を使います。

```
abstract: |
  この文書は、製品の仕様をまとめたものです。

  第 2 章で、**画面の仕様**を説明します。
```

表紙かバージョン履歴があれば、それらの次のページに出します。ページの順は、表紙、バージョン履歴、概要、目次、本文です。どちらもなければ、1 ページ目の題のすぐ下に出します。

概要の中の見出し、図、表、式は、本文の番号と目次に数えません。概要に画像は使えません。

3.6 タイトルを書かないとき

`title` がなければ、次の順に題を決めます。決めた題は、PDF の文書情報、表紙、ヘッダの `{title}` に使います。

1. 最初の H1(`#` で始まる見出し)を題にします。
2. H1 もなければ、ファイル名を題にします。

合本では親 md の H1 だけを見ます。親 md に H1 がなければファイル名になり、章の H1 は使いません。

H1 は、1 つだけでも題にはせず、本文の章として組みます。H1 がなければ、`##` の見出しが章になります。表紙がなければ、決めた題は 1 ページ目に出しません。

`# 文書名` の形で文書名を書くと、文書全体が 1 つの章になります。文書名を題にするときは、`title` に書いて H1 を消し、`##` を `#` に上げます。

3.7 表紙がないとき

表紙を付けない文書では、1 ページ目の先頭に題、副題、著者、日付を出します。題は `title` に書いたものだけを出し、最初の H1 やファイル名から決めた題は出しません。目次もないときは、最初の章を同じページから始めます。

3.8 書き間違えたとき

未知のキーや使えない値があると、警告を出したうえで PDF を作ります。機密表示のキーを書き間違えても黙って表示が消えないように、警告は必ず確認してください。実際の表示では、ファイル名の部分は絶対パスになります。

```
警告: sample.md:7: 未知のキー pubpdf.conficential は無視します
警告: sample.md:8: pubpdf.cover の値 "fancy" は使えません。既定値を使います
```

3.9 frontmatter に書いても効かないもの

表 3-1 にないキーは使いません。Quarto の設定(`format:`や`filters:`など)を書いても無視し、警告を出します。他の人から受け取った原稿で、意図しないプログラムが動かないようにするためです。本文の途中に`---`で囲んだ YAML を書いても、設定としては読みません。その部分は区切り線と文字として PDF に出るので、消してください。

```
警告: sample.md:4: frontmatter のキー filters は使いません
警告: sample.md:7: 本文の途中の YAML は読みません
```

3.10 設定の優先順位

同じ項目を複数の場所で指定したときは、次の順に優先します。

1. 原稿の frontmatter
2. VS Code のワークスペースの設定
3. VS Code のユーザーの設定
4. 既定値

設定のキーの対応は、VS Code での操作にまとめています。

第 4 章　設定一覧

設定は frontmatter の `pubpdf:` の下に書きます。VS Code では、設定パネルからも変更できます。表 4-1 に全項目をまとめます。

表 4-1: pubpdf の設定項目

キー	値	既定	内容
`cover`	`none`、`simple`、`formal`	`none`	表紙の型
`cover-frame`	`true`、`false`	`false`	表紙を二重の線で囲む
`approval-stamp`	`true`、`false`	`false`	承認欄の押印欄
`history-number`	`true`、`false`	`false`	バージョン履歴の項番の列
`history-columns`	列のリスト	なし	バージョン履歴の列の順と見出し
`history-title`	文字列	`バージョン履歴`	バージョン履歴のページの題
`history-blank-rows`	0 から 50	`0`	バージョン履歴の手書き用の空の行
`guides`	`true`、`false`	`true`	記入の指示(`:::{.guide}`)を出すか
`confidential`	`false`、`true`、文字列	`false`	機密表示の文字
`confidential-position`	`none`、`top-right`、`bottom-left`、`both`	`top-right`	機密表示の位置
`logo`	原稿のフォルダの中の画像	なし	表紙のロゴ
`watermark`	文字列	なし	透かしの文字
`header-left`、`header-center`、`header-right`	文字列	左は章の名前	ヘッダの文字
`footer-left`、`footer-center`、`footer-right`	文字列	なし	フッタの文字
`header-rule`、`footer-rule`	`true`、`false`	`false`	ヘッダの下とフッタの上の罫線
`columns`	`1`、`2`、`3`、`4`	`1`	本文の段数
`accent-color`	`"#0a7d5a"` の形の色	なし	テーマの色

キー	値	既定	内容
`table-style`	`grid`、`lines`、`booktabs`、`striped`	`grid`	表の体裁
`code-theme`	`default`、`github`、`atom-one`、`solarized`、`nord`、`dracula`、`monochrome`	`default`	コードの配色
`inline-code-background`	`true`、`false`	`true`	文中のコードに薄い灰色の角丸の背景を付ける
`pdf-standard`	`none`、`a-2b`、`a-3b`、`ua-1`、`a-2b,ua-1`	`none`	PDF の規格
`toc`	`false`、`true`、`1`、`2`、`3`	`2`	目次の有無と深さ
`list-of-figures`	`true`、`false`	`false`	図の一覧
`list-of-tables`	`true`、`false`	`false`	表の一覧
`paper`	`a3`、`a4`、`a5`、`b4`、`b5`、`letter`、`legal`	`a4`	用紙サイズ
`orientation`	`portrait`、`landscape`	`portrait`	用紙の向き
`margin`	`top`、`bottom`、`inner`、`outer` に数値(mm)	用紙で決まる	余白
`font-size`	数値(pt)。5 から 30	`10.5`。A5 は `9.5`	本文の文字の大きさ
`line-spacing`	数値。1 から 3	`1.6`	行間
`first-line-indent`	`true`、`false`	`false`	段落の字下げ
`page-number`	`book`、`right`、`center`、`total`、`none`	`center`。奇数ページ始まりでは `book`	ページ番号の出し方
`page-count`	`continuous`、`roman`	`continuous`	ページの数え方
`numbering`	`none`、`chapter`、`decimal`	`none`	見出しの番号
`heading-format`	番号の形のリスト	なし	階層ごとの見出しの番号の形
`figure-numbering`	`chapter`、`section`、`continuous`	見出しの番号があれば `chapter`	図、表、コード、式の番号
`equation-numbering`	`all`、`labeled`	`all`	番号を入れる式

キー	値	既定	内容
`code-line-numbers`	`true`、`false`	`false`	コードの行番号
`table-fit`	`true`、`false`	`false`	中身の短い表を、中身の幅に合わせる
`code-display`	`auto`、`show`、`fit`、`attach`	`auto`	コードの見せ方(そのまま、縮める、添付ファイル)
`code-fit-min`	4 から 12(pt)	`6`	コードを縮めるときの文字の大きさの下限
`code-attach`	`all`、`named`、`none`	`all`	コードを PDF の添付ファイルにする
`code-attach-note`	文字列	「このコード(10 行)は、添付ファイル … にあります。」の文	添付ファイルに切り替えたコードの場所に出す文
`chapter-break`	`true`、`false`	`true`	章の前の改ページ
`book.odd-start`	`true`、`false`	単一 md は `false`、合本は `true`	奇数ページ始まり
`book.gutter`	数値(mm)	`8`	綴じ代
`chapters`	ファイルのリスト	なし	合本にする章
`appendices`	ファイルのリスト	なし	付録にする章
`bibliography`	原稿のフォルダの中の `.bib`	なし	文献のファイル
`citation-style`	`ieee`、`sist02` など	`ieee`	文献の書式
`footnotes`	`page`、`list`	`page`	脚注をページの下に出すか、番号つきの一覧にまとめるか
`footnotes-title`	文字列	`参考文献`(`bibliography` があれば `注`)	脚注の一覧の題
`output`	原稿のフォルダの中の `.pdf`	`<原稿名>.pdf`	出力先
`font`	フォント名	自動	本文のフォント
`font-heading`	フォント名	本文と同じ	見出しと題のフォント
`font-code`	フォント名	DejaVu Sans Mono	コードのフォント

キー	値	既定	内容
`font-math`	数式用のフォント名	New Computer Modern Math	数式のフォント
`font-paths`	パス、またはそのリスト	なし	フォントの読み込み先

4.1 表紙

`cover`で表紙の型を選びます。

```
pubpdf:
  cover: formal
```

表 4-2: 表紙の型

値	内容
`none`	表紙なし。1 ページ目の先頭に題を出します
`simple`	題、副題、著者、日付だけの表紙
`formal`	文書番号、版数、発行日、作成者の表が付いた表紙

`formal`の表には、frontmatter に書いた項目だけが出ます。

`cover-frame: true`で、表紙を二重の線の枠で囲みます(IPA の様式の表紙の形です)。

4.1.1 ロゴと組織名

`logo`に画像を指定すると、表紙の上部に出します。`simple`では左上、`formal`では中央です。組織名は frontmatter の`organization`に書きます。`simple`では著者の下、`formal`では表の「組織」の行に出ます。

```
organization: 株式会社見本
pubpdf:
  cover: simple
  logo: images/logo.png
```

- 使える画像は、原稿のフォルダの中の png、jpg、gif、svg、webp、pdf です。フォルダの外を指定すると、警告を出して使いません。
- ロゴは縦横比を保って、幅 6cm、高さ 2cm の枠に収めます。
- 表紙がないときは、`logo`は警告を出して使いません。`organization`は 1 ページ目の題の下に出ます。

4.2 機密表示

`confidential`に表示する文字を、`confidential-position`に位置を指定します。

```
pubpdf:
  confidential: 社外秘
  confidential-position: both
```

表 4-3: 機密表示の位置

位置の値	表示する場所
`none`	出さない
`top-right`	右上のみ
`bottom-left`	左下のみ
`both`	右上と左下

- 文字だけを指定すると右上に出します。位置だけを指定すると「Confidential」と出します。`true` と書いても「Confidential」になります。
-
- 表紙、目次、章の開始ページにも出します。自動で入る白紙には出しません。
- 偶数ページでも同じ位置に出します。製本すると、偶数ページの右上は綴じ側に来ます。
- book 形式のページ番号と左下の機密表示が重なる偶数ページでは、番号を外側に置き、その右に機密表示を並べます。

> ⚠ 見た目だけの表示です
>
> 機密表示は文字を印刷するだけで、PDF を保護するものではありません。誰でも開いて読めます。

4.3 透かし

`watermark` に書いた文字を、ページの背景に斜めに薄く出します。

```yaml
pubpdf:
  watermark: 下書き
```

- 文字はページの対角線に沿って置き、長さに合わせて大きさを決めます。
- 表紙を含むすべてのページに出します。自動で入る白紙には出しません。
- 本文の後ろに出るので、色のある表や画像に隠れることがあります。
- 機密表示と同じく、見た目だけの表示です。文字は PDF から検索でき、閲覧を制限するものではありません。

4.4 目次

```yaml
pubpdf:
  toc: 3
```

数字は目次に載せる見出しの深さです。`false` で目次を出しません。`true` は `2` と同じです。見出しが 1 つもない文書には、目次を出しません。

4.5 図と表の一覧

```yaml
pubpdf:
  list-of-figures: true
  list-of-tables: true
```

目次の後に、「図目次」と「表目次」を出します。載るのは、Markdown で書いた番号のある図と表です。横に並べた個々の図は載らず、全体の題だけが載ります。Typst を直接書いて作った図は載りません。該当する図や表が 1 つもなければ、一覧を出しません。

4.6 参考文献

`bibliography` に文献のファイルを指定すると、本文から引用でき、文書の最後に文献の一覧が出ます。書き方は原稿の書き方を参照してください。

一覧を文書の途中に出すときは、出したい位置に `::: {#refs}` と `:::` の 2 行を書きます。脚注を参考文献の一覧にまとめる方法(`footnotes: list`)は、原稿の書き方を参照してください。

```yaml
pubpdf:
  bibliography: refs.bib
  citation-style: sist02
```

表 4-4: 文献の書式

`citation-style` の値	本文の引用の形
`ieee`	[1]
`sist02`	(1)
`springer-vancouver`	(1)
`nature`	上付きの 1
`apa`	(Knuth, 1984)
`chicago-author-date`	(Knuth 1984)
`mla`	(Knuth)

原稿のフォルダの中の `.csl` ファイルを指定して、他の書式を使うこともできます(`citation-style: my-journal.csl`)。

4.7 用紙サイズと向き

```yaml
pubpdf:
  paper: b5
  orientation: landscape
```

`b4` と `b5` は JIS の寸法です(B5 は 182 × 257mm)。余白と本文の文字の大きさは、用紙に合わせて自動で決まります。

`orientation: landscape` で文書全体を横向きにします。一部のページだけを横向きにする方法は、原稿の書き方を参照してください。

4.8 余白

```yaml
pubpdf:
  margin:
    top: 30
    bottom: 25
    inner: 28
    outer: 20
```

単位は mm です。書かなかった辺は既定のままです。既定は上が 27、下が 24 です。左右は用紙で決まり、A3 は 30、B4 は 26、A4、Letter、Legal は 24、B5 は 20、A5 は 16 です。

`inner`と`outer`の意味は、奇数ページ始まりの設定で変わります。

表 4-5: 内側と外側の余白

奇数ページ始まり	`inner`	`outer`
オン	綴じる側(綴じ代はこの値に足します)	綴じない側
オフ	左に固定	右に固定

余白や綴じ代が大きすぎて、本文の幅か高さが 50mm 未満になるときは、警告を出して既定の値で組みます。一部のページだけが横向きの文書では、用紙の短い辺でも確かめます。

4.9 段組み

`columns`で本文の段数を指定します。本文の一部だけを段組みにする方法は、原稿の書き方を参照してください。

```yaml
pubpdf:
  columns: 2
```

- 表紙、履歴、概要のページ、目次は 1 段のままで、本文から段組みにします。
- 前付けがない文書では、題と概要を段の上に通して出し、同じページから段組みを始めます。
- 新しいページから始まる章の題は、段の上に通して出します。`chapter-break: false`では段の中に出します。
- 長い概要(版面の 3 分の 1 ほどを超えるもの)は、題の下ではなく概要のページに出します。
- 独立した式は折り返さないので、段の幅より長い式は段からはみ出します。
- 最後のページでは、左の段から順に埋めるので、段の長さは揃いません。
- 段の間は 6mm です。段の幅が 30mm より狭くなるときは、警告を出して 1 段で組みます。

4.10 文字の大きさと行間

```yaml
pubpdf:
  font-size: 11
  line-spacing: 1.8
  first-line-indent: true
```

`font-size` は本文の文字の大きさです。見出し、コード、表、数式も比例して変わります。ヘッダ、フッタ、表紙の題は変わりません。

`line-spacing` は、文字の大きさに対する行送りの倍率です。段落の間隔も行間に合わせて広がります。表とコードの行間は変わりません。

`first-line-indent: true` にすると、本文の段落の先頭を 1 文字下げます。リスト、引用、囲み、表、図の題、脚注は下げません。

4.11 ページ番号

`page-number` で番号の出し方を選びます。表紙には番号を出しません。

表 4-6: ページ番号の出し方

値	出し方
`book`	book 形式。紙の上で奇数枚目は右下、偶数枚目は左下
`right`	右下に固定
`center`	中央に固定
`total`	中央に「3 / 12」の形で出す。分母は最後のページの番号で、白紙も数えます
`none`	出さない

`page-count` でページの数え方を選びます。

```yaml
pubpdf:
  page-count: continuous
```

表 4-7: ページの数え方

値	数え方	表紙、履歴、目次、第 1 章の番号の例
`continuous`	表紙を 1 ページ目として、最後まで通して数える	1(非表示)、2、3、4
`roman`	本文の前をローマ数字で数え、本文を 1 から数え直す	i(非表示)、ii、iii、1

自動で入る白紙も 1 ページとして数えます。番号は出しません。`roman` と `total` を組み合わせると、本文の前には分母を出さず、分母は本文のページ数になります。表紙のすぐ次から本文が始まる奇数ページ始まりの文書では、`roman` でも通して数えます。

4.12 ヘッダとフッタの文字

ヘッダとフッタの左、中央、右に文字を入れられます。

```yaml
pubpdf:
  header-left: '{title}'
  header-right: '{chapter}'
  footer-left: '{organization}'
  footer-center: '{page} / {total}'
```

`{名前}`は、次の値に置き換わります。`{`で始まる値は YAML の決まりで引用符が必要です。

表 4-8: ヘッダとフッタに書ける値

書き方	置き換わる値
`{title}`、`{subtitle}`、`{author}`、`{organization}`、`{date}`、`{version}`、`{doc-number}`	frontmatter の値
`{chapter}`	そのページの章の名前。番号があれば番号も出します
`{page}`	ページ番号。`page-count`の数え方に従います
`{total}`	最後のページの番号

- `{title}`などの Markdown として読む値は、記号を除いて出します。
- `{page}`をどこかに書くと、`page-number`による自動のページ番号は出しません。`{total}`だけなら自動の番号も出します。
- `header-left`を指定しないときは、これまでどおり章の名前を出します。章の名前を消すときは`header-left: ""`と書きます。
- 文字は Markdown として読まず、書いたとおりに出します。
- 表紙と、自動で入る白紙には出しません。
- `header-rule: true`でヘッダの下に、`footer-rule: true`でフッタの上に罫線を引きます。表紙には引きません。
- 機密表示やページ番号と同じ場所を指定すると、少し空けて並べます。
- 使えない`{名前}`は、警告を出して文字のまま出します。値のない`{version}`などは、警告を出して空にします。

4.13 見出しと図表の番号

`numbering`で見出しの番号の形を選びます。

```yaml
pubpdf:
  numbering: chapter
```

表 4-9: 見出しの番号の形

値	章	節
`none`	番号なし	番号なし
`chapter`	第 1 章	1.1
`decimal`	1	1.1

`heading-format`で、階層ごとに番号の形を指定できます。設計書でよく使う「1. → 1.1 → (1) → ①」の形は、次のように書きます。`numbering`が`chapter`か`decimal`のときだけ使います。

```yaml
pubpdf:
  numbering: decimal
  heading-format: ["1.", "1.1", "(1)", "①"]
```

表 4-10: 番号の形の書き方

形の例	出る番号
`1.`	1. 2. 3.
`第1章`	第 1 章 第 2 章
`1.1`、`1-1`	2.1 2.2 のように、上の階層の番号を含む
`(1)`、`①`、`(ア)`、`a)`	自分の階層の番号だけ

- 形の中の数える記号(`1`、`a`、`A`、`i`、`I`、`ア`、`あ`、`一`、`①`など)の数だけ、下の階層から番号を使います。`1.1`なら上の階層と自分、`(1)`なら自分だけです。
- 指定より深い階層は、最後の形を使います。
- 付録の章は「付録 A」です。上の階層の番号を含む形では、章の番号を A の形にします(`1.1`なら A.1)。
- 形は 6 個まで、1 つ 20 文字までです。数える記号のない形は使えません。

`figure-numbering`で、図、表、コード、定理、式の番号の振り方を選びます。

表 4-11: 図表の番号の振り方

値	例	振り方
`chapter`	図 1-1、表 2-1、(1-1)	章ごとに振り直す
`section`	図 1.1-1、表 2.3-1、(1.1-1)	節ごとに振り直す。章の最初の節より前は「図 1-1」
`continuous`	図 1、表 2、(3)	文書全体を通して数える

既定は、見出しの番号があれば`chapter`、なければ`continuous`です。見出しの番号がないときに`chapter`か`section`を指定すると、警告を出して通し番号にします。

序文や`{.unnumbered}`を付けた章は、番号のある章の外として扱います。その中の節にも番号を付けません。図と式には章の番号を付けず、番号を振り直さずに続けて数えます。

本書は`chapter`で組んでいます。

4.14 式の番号

`equation-numbering`で番号を入れる式を選びます。

表 4-12: 番号を入れる式

値	番号を入れる式
`all`	すべての独立した式。`\notag`と`\nonumber`を書いた式を除きます
`labeled`	`{#eq-名前}`を付けた式だけ

書き方は原稿の書き方を参照してください。

4.15 コードの行番号

`code-line-numbers: true`にすると、すべてのコードに行番号を付けます。コードごとに指定する方法は、原稿の書き方を参照してください。

4.16 コードの見せ方

`code-display` で、コードを本文にどう出すかを選びます。既定の `auto` は、コードの大きさで切り替えます。

表 4-13: コードの見せ方

`code-display` の値	見せ方
`auto` (既定)	1 ページに入ればそのまま(途中で分けない)。入らなければ文字を縮めて 1 ページに収め、下限まで縮めても入らなければ付録へ移す
`show`	そのまま。ページの途中から始まったコードも、ページをまたいで分ける
`fit`	1 ページに入ればそのまま(途中で分けない)、入らなければ文字を縮めて 1 ページに収める。下限まで縮めても入らなければ、そのままページをまたぐ
`appendix`	本文には載せず、付録へ移す
`attach`	本文には載せず、PDF の添付ファイルにする

```yaml
pubpdf:
  code-display: auto
  code-fit-min: 7     # 縮める下限(pt)。既定は 6
```

- 付録へ移したコードは、文書の最後に「付録 A コード」の章を作って、そのままの大きさで載せます(ページをまたぎます)。付録のある文書では、付録の最後の章になります。コードごとに「A.1 app.py」のような節にし、名前のないコードは「コード 1」とします。節には、本文の置き場所(ページ)へのリンクを添えます。
- 本文には「このコード(300 行)は、付録の A.1 に載せています。」の枠を出します(A.1 は付録の節へのリンク)。この文は `code-appendix-note` で書き換えられます。`{ref}` は付録の節の番号、`{name}` はファイル名、`{lines}` は行数に置き換わります。付録の章の題は `code-appendix-title` で変えられます。コードごとには `note="..."` で文を上書きします。

  ```yaml
  pubpdf:
    code-appendix-title: ソースコード
    code-appendix-note: "{name}({lines} 行)は {ref} を見てください。"
  ```

- 添付ファイルにしたコード(`attach`)は、本文に「このコード(300 行)は、添付ファイル big.py にあります。」の枠を出します。この文は `code-attach-note` で書き換えられます(`{name}` は添付の名前)。
- `auto` と `fit` では、1 ページに入るコードを途中で分けません。ページの残りに入らなければ、丸ごと次のページへ送ります。前のページの下が空くことがあります。
- コードの直前の短い段落(「次のように書きます。」など)は、コードと一緒に次のページへ送ります。ただし、版面の 8 割を超えるコードの前の段落は、見出しと段落だけのページができないように、前のページに残します。
- コードごとに選ぶときは、 `{.python .show}` のように `.show` 、 `.fit` 、 `.appendix` 、 `.attach` 、 `.auto` を書きます(原稿の書き方)。

- 1.6 までの `auto` は、1 ページに入らないコードを添付ファイルに切り替えていました。PDF の添付ファイルは、会社によってはメールで止められるなどして受け取れないので、1.7 から付録へ移すようにしました。
- 1.1 の `code-fit` も使えます。 `true` は `fit`、 `false` は `show` と同じです。
- PDF/A-2b では添付ファイルを付けられないので、 `attach` は `show` として組みます。

4.17 コードの添付

表示したコードも、PDF の添付ファイルにできます(`attach` で添付ファイルにしたコードは、いつも添付します)。PDF の表示からコピーすると、ビューアによっては字下げの空白が 1 つにまとまりますが、添付ファイルなら書いたとおりの文字で取り出せます。Acrobat Reader では左の「添付ファイル」(クリップのアイコン)、Firefox ではサイドバーの添付ファイルの一覧から開けます。

添付ファイルのある PDF は、会社によってはメールやファイルの受け渡しで止められます。既定は添付しません(1.6 までの既定は `all` でした)。渡す相手が受け取れることを確かめてから使ってください。

表 4-14: コードの添付

`code-attach` の値	添付するコード
`none` (既定)	表示したコードは添付しない
`all`	表示したすべてのコード
`named`	表示したコードのうち、ファイル名を書いたものだけ

- ファイル名を書いたコード(`python: app.py`)と、ファイルを参照したコード(`include="src/app.py"` は `app.py`)は、その名前で添付します。名前のないコードは `code-1.py` のように、言語名から拡張子を決めます。同じ名前が重なると、`app-2.py` のように番号を付けます。
- PDF/A-2b では添付ファイルを付けられないので、添付しません。PDF/A-3b と PDF/UA では添付します。
- タブは、Markdown を読むときに空白に置き換わります。添付ファイルでも空白です。

4.18 章の前の改ページ

`chapter-break: false` にすると、章の前で改ページしません。短い文書で間延びするときに使います。ただし、奇数ページ始まりのときは、章は必ず新しいページから始まります。

4.19 色と体裁

4.19.1 テーマの色

`accent-color` で、題、見出し、章と表紙の線、リンクと参照、表の見出しの色を変えます。`#` は YAML ではコメントの印なので、引用符で囲みます。

```yaml
pubpdf:
  accent-color: "#0a7d5a"
```

4.19.2 表の体裁

`table-style` で、本文の表の罫線を選びます。バージョン履歴の表にも効きます。表紙の表は変わりません。

表 4-15: 表の体裁

値	体裁
`grid`	格子。見出しの行を塗ります
`lines`	横線だけ。見出しの行を塗ります
`booktabs`	表の上下と見出しの下の線だけ。論文でよく使う形です
`striped`	線なしで、行ごとに色を分けます

`table-fit: true` にすると、中身の短い表を版面いっぱいに広げず、中身の幅に合わせます。表ごとの指定(`.fit` 、 `.full`)は、原稿の書き方を参照してください。

4.19.3 コードの配色

`code-theme` で、コードブロックの背景と色分けを選びます。

表 4-16: コードの配色

値	配色
`default`	既定
`github`	GitHub
`atom-one`	Atom One
`solarized`	Solarized。背景が淡い黄色
`nord`	Nord。背景が暗い
`dracula`	Dracula。背景が暗い
`monochrome`	白黒

背景が暗い配色では、文中のコード(`` `code` ``)は色分けせず、本文と同じ色で出します。

4.19.4 文中のコードの背景

文中のコード(`` `code` ``)には、薄い灰色の角丸の背景を付けます。背景は文字の外へ広げるだけなので、行の間は変わりません。長いコードは、これまでどおり途中で折り返します。背景を付けないときは `inline-code-background: false` にします。

```
pubpdf:
  inline-code-background: false
```

パスと URL の書き方(`{.path}`)の枠と背景は、この設定によらず付きます。

4.20 フォント

本文、見出し、コード、数式のフォントを別々に指定できます。

```yaml
pubpdf:
  font: BIZ UDPGothic
  font-heading: Noto Sans JP
  font-code: Consolas
  font-math: Cambria Math
```

本文のフォントを指定しないときは、インストールされているフォントから、次の順に最初に見つかったものを使います。

1. Noto Sans JP
2. Noto Sans CJK JP
3. BIZ UDPGothic
4. Yu Gothic
5. Meiryo

欧文だけのフォントを指定しても、日本語の文字はこの順に見つかったフォントで補います。リストで書くと、前のフォントにない文字を次のフォントで補います。

```yaml
pubpdf:
  font:
    - Times New Roman
    - BIZ UDPGothic
```

指定したフォントが見つからないときは、警告を出して別のフォントで組みます。

4.20.1 フォントの読み込み

OS に入れていないフォントも使えます。`font-paths` にフォントのあるフォルダかフォントファイルを指定し、`font` などでフォント名を指定します。相対パスは原稿のフォルダが基準です。読み込めるのは ttf、otf、ttc、otc です。

```yaml
pubpdf:
  font-paths:
    - fonts-custom
    - assets/MyFont-Regular.otf
  font: My Font
```

原稿と同じフォルダの `fonts/` は、指定しなくても読み込みます。VS Code では設定パネルのフォント名の欄に、読み込めるフォントが候補として出ます。ファイルを指定したときは、同じフォルダの他のフォントも候補に出ますが、使えるのは指定したファイルだけです。

4.20.2 数式のフォント

`font-math` には数式用のフォントだけが使えます。数式用でないフォントを指定すると、大きな括弧や総和の記号が崩れます。Windows の Cambria Math のほか、Fira Math、STIX Two Math、Libertinus Math などを `font-paths` で読み込めば使えます。

4.21 PDF の規格

`pdf-standard` で、長期保存やアクセシビリティの規格に沿った PDF を作ります。

```
pubpdf:
  pdf-standard: a-2b
```

表 4-17: PDF の規格

値	規格	用途
`a-2b`	PDF/A-2b	長期保存
`a-3b`	PDF/A-3b	長期保存
`ua-1`	PDF/UA-1	アクセシビリティ(読み上げなど)
`a-2b,ua-1`	PDF/A-2b と PDF/UA-1	両方

- 組むときに Typst が規格の条件を確かめ、合わない内容があれば組まずに止めます。
- PDF/A と PDF/UA には、PDF の画像を使えません。PNG か SVG にしてください。
- PDF/UA では見出しの段を飛ばせません(`#` の次に `###` など)。`skipped from heading level` や `the first heading must be of level 1` と出たら、見出しの段を直してください。
- 外部の検証ツールでの確認が必要なときは、`quarto install verapdf` で veraPDF を入れます。入っていれば、Quarto が組んだ後に検証します。Pubpdf の開発では veraPDF での確認はしていません。

PDF/UA では、画像と式に代替テキスト(読み上げ用の文)が必要です。次の順に自動で入れます。

表 4-18: PDF/UA の代替テキスト

対象	代替テキスト
画像	`fig-alt` の値、`![説明](画像)` の説明の順。どちらもなければ「図」とし、警告を出します
Mermaid の図	`%%| label: fig-名前` と `%%| fig-cap:` で書いた題。なければ「図」とし、警告を出します
式	式の TeX の文字。文献の題の中の式は「数式」
ロゴ	組織名。なければ「ロゴ」

`{=typst}` で直接書いた画像は「図」、式は「数式」になります。説明を付けるときは、式の前の行に `#set math.equation(alt: "説明")` と書きます。

第 5 章　本として組む

両面印刷や製本を前提にした組み方と、複数の md を 1 冊にまとめる方法を説明します。

5.1　奇数ページ始まり

本では、章を右ページ(奇数ページ)から始めるのが普通です。`odd-start`をオンにすると、章の手前の偶数ページが余るときだけ、白紙を 1 枚入れます。

```yaml
pubpdf:
  book:
    odd-start: true
```

図 5-1 は、表紙と目次のある本の例です。表紙の次の p.2 が目次で、p.4 が自動で入った白紙です。第 2 章は p.5 から p.6 まで続き、偶数ページで終わるので、第 3 章の前に白紙は入りません。

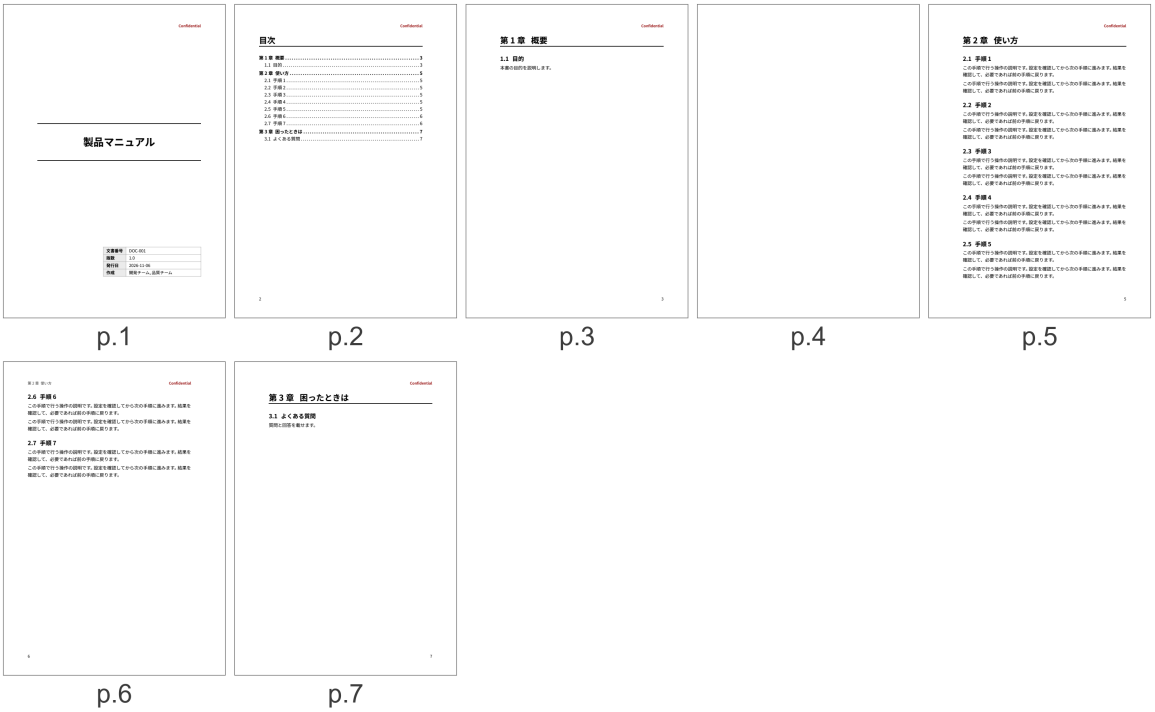


図 5-1: 奇数ページ始まりで組んだ本の例。ページの下の番号は紙の上の順番です

オンにしたときの動作は、次のとおりです。

- 本文の開始と各章を奇数ページから始めます。
- 表紙の次には白紙を入れず、次のページからバージョン履歴か目次を始めます。目次が奇数ページで終わるときは、本文の前に白紙が入ります。バージョン履歴と目次の間には白紙を入れません。
- 白紙には、ヘッダ、ページ番号、機密表示を出しません。白紙も 1 ページとして数えます。
- 奇数かどうかは、紙の上の順番で決めます。ページの番号も紙の上の順番と同じなので、番号と左右はずれません。`page-count: roman`でも、本文の番号は奇数ページで 1 から始まるので、ずれません。

バージョン履歴も目次もなく表紙があるときは、表紙の次のページ(p.2)から最初の章を始めます。このときだけ最初の章は偶数ページから始まります。2 つ目以降の章は奇数ページから始まります。

オフにすると白紙を入れません。

5.2 綴じ代

綴じる側の余白を広げます。単位は mm で、既定は 8 です。奇数ページ始まりのときだけ効きます。

```yaml
pubpdf:
  book:
    odd-start: true
    gutter: 10
```

奇数ページは左の余白、偶数ページは右の余白が広くなります。ページ番号の既定は book 形式で、綴じない側に出ます。

5.3 複数の md を 1 冊にする(合本)

親となる md を 1 つ作り、frontmatter の`chapters`に章のファイルを順に並べます。これを合本と呼びます。

```yaml
---
title: 製品マニュアル
author: 開発チーム
pubpdf:
  cover: formal
  numbering: chapter
  chapters:
    - 01-intro.md
    - 02-design.md
    - appendix/03-faq.md
---
```

親の md を開いて PDF を作ります。VS Code では、親の md が同じワークスペースのフォルダにあれば、章の md を開いた状態で実行しても本全体を組みます。ファイルだけを開いたときや、親の md がワークスペースの外にあるときは、その章だけを組みます。

合本では、奇数ページ始まりが既定でオンになります。

5.3.1 親 md の本文

親 md に本文があれば、序文として最初の章の前に入れます。序文には章番号が付きません。見出しがなければ「序文」を補います。本文がなければ序文は作らず、最初の章から始まります。

5.3.2 付録

`appendices` に並べたファイルは、章の後に付録として組みます。

```yaml
pubpdf:
  numbering: chapter
  chapters:
    - 01-intro.md
  appendices:
    - appendix-a.md
    - appendix-b.md
```

見出しは「付録 A」「A.1」、図と表は「図 A-1」、式は「(A-1)」の形になります。 `@sec-名前` で参照すると「付録 A」と出ます。見出しに番号のない文書(`numbering: none`)では、付録の章だけに「付録 A」を付けます。節には番号を付けず、図と表は通し番号のままです。

単一の md で付録を作るときは、最初の付録の章の見出しに `{.appendix}` を書きます。そこから後の章が付録になります。H1 が 1 つだけの文書では `##` の見出しが章なので、 `##` の見出しに書きます。

```
# 用語集 {.appendix}
```

文献の一覧は、付録の後に出します。

5.3.3 章 md に書くこと

書名や設定は親 md を正とします。章の md で使われるのは、章題の決定に関わる部分だけです。

章の md の内容	章題
H1 がある	H1
frontmatter の `title` だけがある	`title`
両方ある	H1。警告を出します
どちらもない	ファイル名。警告を出します

章の md に `pubpdf:` を書いても無視し、警告を出します。 `chapters` に指定できるのは、拡張子が `.md` 、 `.markdown` 、 `.qmd` のファイルだけです。

原稿のフォルダの外のファイルを指定すると、そのまま取り込んで警告を出します。受け取った原稿を組むときは、意図しない文書が PDF に入っていないかを、この警告で確かめてください。同じファイルを 2 回指定すると、最初の指定だけを使って警告を出します。

5.3.4 章をまたぐリンク

相対パスで別の章を指せば、PDF の中のリンクになります。書き方は原稿の書き方を参照してください。

5.3.5 画像

章ごとに別のフォルダに置いた画像も、その章の md からの相対パスのまま表示されます。

第 6 章　原稿の書き方

本文は通常の Markdown で書けます。この章では、書き方と組み上がりを並べて示します。

6.1　段落と強調

段落の途中で改行しても、日本語の間に余計な空白は入りません。**太字**、`インラインコード`、脚注[1]が使えます。

```
**太字**、`インラインコード`、脚注[^1]が使えます。

[^1]: 脚注はページの下に出ます。
```

同じ箇所で複数の脚注を引くときは、`[^1,2]`のように 1 つの括弧に並べます(区切りは`,`か`、`)。ページの下の脚注では番号を「1,2」と並べ、脚注を参考文献の一覧にするときは「[1, 2]」、3 つ以上続く番号は「[1–3]」にまとめます。グリッド表の中では、縦線がずれないように`[^1][^2]`と書いてください。

`*斜体*`の記法も書けますが、斜体の字形がないフォントでは見た目が変わりません。日本語の強調には太字を使ってください。

文中のコードには、薄い灰色の角丸の背景が付きます。背景は文字の外へ広げるだけなので、行の間は変わりません。背景を付けないときは、設定の`inline-code-background`を`false`にします。

`:smile:`や`:+1:`のような絵文字の名前は、絵文字(😄 👍)にします。コードの中ではそのままです。

6.2　脚注を参考文献の一覧にする

設定を`footnotes: list`にすると、脚注をページの下ではなく、番号つきの一覧にまとめます。Zenn の脚注と同じ書き方で、引用した文献の一覧を作れます。

```
pubpdf:
  footnotes: list
```

```
組版の教科書[^tex]と、設計書の様式[^ipa]を参照しました。
もう一度、組版の教科書[^tex]を引きます。

[^tex]: D. E. Knuth, The TeXbook, Addison-Wesley, 1984.
[^ipa]: IPA, 設計書の様式, 2020.

::: {#refs}
:::
```

- 本文には「[1]」の形で番号を出し、選ぶと一覧へ移ります。番号は本文に出てくる順です。
- 同じ脚注を何度引いても、同じ番号になります。

[1]脚注はページの下に出ます。

- `[^tex,ipa]`のように並べて引くと「[1, 2]」の形になります。
- 一覧は、`::: {#refs}`と`:::`の 2 行を書いた位置に出ます。書かなければ文書の最後に出ます。
- 一覧の題は「参考文献」です。`footnotes-title`で変えられます。`.bib`の文献(文献の引用)も使う文書では「注」になり、文献の一覧はその後に続きます。
- 合本では、すべての章の脚注を 1 つの一覧にまとめます。章ごとに同じ名前(`[^1]`など)を使っても混ざりません。
- `::: {#refs}`を書いても、脚注を一覧にせず、`.bib`の文献もないときは、警告を出します。

6.3 文字を右や中央に寄せる

`::: {.right}`の囲みの中は右に、`::: {.center}`の囲みの中は中央に寄せます。中には Markdown をそのまま書けます。`:::right`、`:::center`と短く書くこともできます。

```
::: {.right}
2026 年 11 月 7 日\
**開発チーム**
:::

::: {.center}
以上
:::
```

2026 年 11 月 7 日
開発チーム

以上

行の途中で改行するときは、行末に`\`を書きます。段落を分けると、段落ごとに寄せます。

- 寄せるのは段落の単位です。文の一部だけを寄せることはできません。
- 囲みの中の図も一緒に寄ります。コードの行は左のまま、独立した式は中央のままです。表の中の文字は、表の列の揃え方に従います。
- リスト、引用、他の囲みの中にも書けます。

6.4 段組み

`::: {.columns}`の囲みの中を 2 段に組みます。`count`で段数を指定できます。文書全体の段組みは`columns`の設定です。

```
::: {.columns}
ここに書いた文章は、左右の段に分かれて流れます。

段の長さは揃います。
:::

::: {.columns count=3}
3 段に組みます。段数は 2 から 4 までです。
:::
```

文章だけのときは、段の長さを揃えます。図、表、コード、囲みなど途中で分けられないものを含むときは、中身が隠れないよう段の高さに余裕を取るので、右の段が短くなることがあります。残りのページに入らないときは、残りが少なければ次のページへ送り、十分あれば段の長さを揃えずにページをまたいで流します。その場合、最後のページの段は左から順に埋まります。段の幅が 30mm より狭くなるときは、段数を減らします。

左右の幅を指定して並べるときは、`.column` の囲みを並べます。幅を書かない列は、残りの幅を等分します。

```
::: {.columns}
::: {.column width="30%"}
左の列です。
:::

::: {.column width="70%"}
右の列です。
:::
:::
```

- `.column` の囲みと他の内容を混ぜると、警告を出して `.column` の幅を使いません。
- 中で改ページと横向きはできません。警告を出して無視します。
- 中の見出しは目次に載りません。

6.5 文字の修飾

表 6-1: 文字の修飾の書き方

書き方	組み上がり
`~~取り消し~~`	~~取り消し~~
`H~2~O`	H_2O
`x^2^`	x^2
`[下線]{.underline}`	下線
`[蛍光ペン]{.mark}`	蛍光ペン

6.6 パスと URL

共有フォルダのパスや長い URL は、文中のコードに `{.path}` を付けて書きます。背景と細い枠を付け、`\` と `/` の後ろで改行できるようにします。文字は書いたとおりに出します(`\` は消えません)。`http://` と `https://` で始まるものは、リンクにします。

```
`\\fileserver\share\設計\2026\基本設計書.docx`{.path} に置きます。
`https://example.com/docs/manual/index.html`{.path} を見てください。
```

`\\fileserver\share\設計\2026\基本設計書.docx` に置きます。`https://example.com/docs/manual/index.html` を見てください。

枠と背景は、`inline-code-background` の設定によらず付きます。

6.7 HTML のタグ

Qiita、Zenn、GitHub 向けに書いた HTML のタグのうち、次のものは体裁に変えます。開きと閉じのタグが 1 つの段落の中で対になっているものだけを変えます。対にならないタグは、タグを除いた文字だけを出します。

表 6-2: HTML の文中のタグ

書き方	組み上がり
`<b>太字</b>`、`<strong>`	**太字**
`<i>斜体</i>`、`<em>`	斜体
`<u>下線</u>`、`<ins>`	下線
`<s>取り消し</s>`、`<del>`	~~取り消し~~
`<mark>蛍光ペン</mark>`	蛍光ペン
`<code>code</code>`	`code`
`<small>小さい字</small>`	小さい字
`<kbd>Ctrl</kbd>+<kbd>C</kbd>`	Ctrl+C
`x<sup>2</sup>`、`H<sub>2</sub>O`	x^2、H_2O
`<a href="https://example.com">リンク</a>`	リンク
`<font color="red">赤い字</font>`	赤い字

`<font color>`の色は、`red`などの名前か`#ff0000`の形で書きます。

ブロックのタグは、次のように変えます。

- `<details><summary>題</summary>`から`</details>`までは、`:::details 題`と同じ灰色の囲みにします(節 6.13)。
- `<div align="center">`、`<p align="center">`、`<center>`は中央に、`align="right"`は右に寄せます。
- `<blockquote>`は引用、`<dl>`と`<dt>`と`<dd>`は用語と説明の並び、`<pre>`はコードにします。
- `<img src="..." width="...">`は画像にします(節 6.18)。
- ほかのタグ(`<h2>`など)は、タグを除いた文字を段落として出します。`style`属性の CSS は、一部を除いて効きません。`<iframe>`などの埋め込みは出しません。

6.8 リスト

- 箇条書きは`-`で書きます。
  - 字下げすると入れ子になります。

1. 番号付きのリストは`1.`で書きます。
2. 番号は自動で振られます。

☒ チェック付きのリストは`- [x]`で書きます。

☐ 未完了の項目は`- [ ]`で書きます。

用語と説明の組は、次の行を`:`で始めます。

```
用語
: 用語の説明です。
```

用語
: 用語の説明です。

6.9 表

Markdown の表をそのまま書けます。列の幅は内容の量に合わせて自動で配分します。幅を決めたいときは、表の題に`tbl-colwidths`で列の幅の比を書きます(例: `: 題 {#tbl-名前 tbl-colwidths="[20,30,50]"}`)。ただし、折り返せない英数字(`No.` 、日付、ID など)が収まる幅より狭くはしません。セルの左右の余白があるので、`No.`の列は 4 と書いても 8% ほどになります。グリッド表の罫線の長さは、幅の指定には使いません。

全部のセルが空の行も、文字のある行と同じ高さで組みます。記入用の空の行に使えます。

表の直後か直前に`:`で始まる行を書くと、表の題になり、番号が自動で振られます。番号の振り方は、設定の`figure-numbering`で選べます。

```
| 項目 | 説明 |
|---|---|
| 表紙 | 3 種類から選べます |

: 表の題
```

表 6-3: 表の題

項目	説明
表紙	3 種類から選べます

本文から参照するときは、題の行に`{#tbl-名前}`を付けます。`@tbl-名前`と書くと番号に置き換わります(例: 表 6-4)。

```
: 参照する表の題 {#tbl-sample}

@tbl-sample を参照してください。
```

表 6-4: 参照する表の題

項目	説明
機密表示	位置を 4 通りから選べます

題のない表には番号が付きません。長い表はページをまたぎ、次のページで見出し行を再び表示します。

セルを結合するときは、罫線を文字で描く書き方(グリッド表)を使います。縦の罫線の位置を揃えてください。全角の文字は半角 2 文字分として数えます。

```
+------+-------+
| 結合したセル |
+======+======+
| a    | b     |
+------+-------+
| 結合したセル |
+------+-------+

: セルを結合した表
```

表 6-5: セルを結合した表

結合したセル	
a	b
結合したセル	

見出しの行は、太字にして色を付けます。グリッド表で`+===+`の行を書かなければ、見出しのない表になり、1 行目も普通のセルとして組みます。見出しが 2 段以上でも、すべての段に同じ体裁が付きます。

グリッド表では、全角の文字を 2 文字分として罫線を揃えます。`○`、`×`、`→`、`℃`、`①`などの記号は、エディタによって 1 文字分にも 2 文字分にも見えます。どちらで揃えても、縦線を罫線の位置に合わせて読みます。合わせられない行があると、警告を出します。

列の多い表は、表の題に`.small`(85%)か`.x-small`(75%)を書くと、その表の文字を小さくできます。列の幅の配分も、小さい文字に合わせます。

```
: 画面入出力項目一覧 {#tbl-items .small}
```

表は、ふつうは版面の幅いっぱいに広げます。中身の短い表は、題に`.fit`を書くと、中身の幅に合わせて中央に置きます。文書のすべての表をそうするには、設定の`table-fit: true`を使います。このとき、広げたい表には`.full`を書きます。

```
: 定理の種類 {#tbl-kinds .fit}
```

- 中身の幅に合わせても版面の 8 割を超える表は、版面の幅に広げます。
- 列をまたいで結合したセルのある表と、`tbl-colwidths`を書いた表は、版面の幅のままです。

6.9.1 セルの網掛けと改行

セルの中身全体を `[文字]{.shade}` と書くと、そのセルに網掛けをします。帳票の形の表で、項目名の列や区切りの行に使います。 `fill="#ccccff"` で色を指定できます。

```
+------------------+--------------------+
| [PU ID]{.shade}  | PU-0101            |
+------------------+--------------------+
| [処理内容]{.shade}                    |
+---------------------------------------+
| 温度を読む\                           |
| PV を更新する                         |
+---------------------------------------+
```

表 6-6: 帳票の形の表

PU ID	PU-0101
処理内容	
温度を読む PV を更新する	

セルの中で改行するときは、`<br>` を書きます。グリッド表では、行の末尾の `\` でも改行できます。 `<br>` は本文の段落でも使えます。

```
| 項目 | 備考 |
|---|---|
| 郵便番号 | 会社の郵便番号<br>記述形式: 999-9999 |
```

パイプ表のセルで箇条書きにするときは、`<br>` で区切った行を `-` で始めます。 `1.` で始めると番号つきのリストになります(番号つきは 2 行以上のときだけ。「1. 概要」のような 1 行のセルはリストにしません)。リストの前後に文も書けます。HTML の `<ul><li>` と `<ol><li>` も同じように箇条書きになります。入れ子のリストはグリッド表で書いてください。

```
| 項目 | 内容 |
|---|---|
| 機能 | 次の 2 つ<br>- 起動する<br>- 設定を読む |
| 手順 | 1. 起動<br>2. 設定<br>3. 終了 |
| HTML | <ul><li>一つ目</li><li>二つ目</li></ul> |
```

表 6-7: セルの中の箇条書き

項目	内容
機能	次の 2 つ • 起動する • 設定を読む
手順	1. 起動 2. 設定 3. 終了
HTML	• 一つ目 • 二つ目

グリッド表では、セルの中にふつうの Markdown のリストを書けます(入れ子もできます)。

6.9.2 HTML の表

`<table>` で書いた HTML の表も、表として組みます。`rowspan` と `colspan` のセル結合、`<br>` の改行、`<b>` の太字、`style="background-color: #ccccff"` のセルの色が使えます。`<th>` だけの先頭の行は、見出しの行になります。本文の行の `<th>` は、行の見出しとして太字と網掛けになります。Markdown の表と同じく、直前か直後の `: 題 {#tbl-名前}` の行が表の題になり、`tbl-colwidths` で列の幅も指定できます。

```
<table>
<tr><th rowspan="2">機能</th><th colspan="2">エンティティ</th></tr>
<tr><th>顧客</th><th>受注</th></tr>
<tr><td>顧客の登録</td><td>C</td><td></td></tr>
<tr><td>受注の登録</td><td>R</td><td>C</td></tr>
</table>

: CRUD 表 {#tbl-crud}
```

表 6-8: HTML で書いた表

機能	エンティティ	
	顧客	受注
顧客の登録	C	
受注の登録	R	C

- 表の中の `<img>` の画像は出しません。警告を出して取り除きます。
- 引用やリストの中の HTML の表は、表になりません。

6.10 コード

コードブロックは、言語名を付けると色分けされます。長い行は折り返します。区切りのない長い文字列(URL やハッシュ値など)も途中で折り返します。

言語名の後に`:`とファイル名を続けると、ファイル名を左上に出します(Qiita や Zenn と同じ書き方です)。`:`の後の空白はなくても構いません。

````
```python: app.py
def add(a, b):
    # 合字にしないので、== や >= は入力したとおりに印刷されます
    assert a >= 0 and b >= 0
    return a + b
```
````

app.py

```python
def add(a, b):
    # 合字にしないので、== や >= は入力したとおりに印刷されます
    assert a >= 0 and b >= 0
    return a + b
```

言語名がなければ、色分けせずにファイル名だけを出します(```` ```:memo.txt ````)。ファイル名の帯がページの下に 1 つだけ残ることはなく、少なくともコードの最初の数行と一緒に置きます。Quarto の書き方の```` ```{.python filename="app.py"} ````も同じ結果になります。

言語名の後に空白と別の語を続けた```` ```js title ````は、言語名だけを使います。```` ```js title="app.js" ````の`title`は、ファイル名として左上に出します。

言語名を`diff`にすると、差分の書き方になります。`+`で始まる行に薄い緑、`-`で始まる行に薄い赤、`@@`の行に薄い青の背景を付けます。行の先頭の記号はそのまま出します。Zenn の```` ```diff js ````と Qiita の```` ```diff_javascript ````のように言語名も書くと、その言語で色分けします。ファイル名、行番号、縮小、添付と一緒に使えます。添付ファイルの中身は、書いたとおり(記号つき)です。

````
```diff js:app.js
 const a = 1;
-const b = 2;
+const b = 3;
```
````

app.js

```diff
 const a = 1;
-const b = 2;
+const b = 3;
```

言語名のない```` ```diff ````では、`>`で始まる行も追加、`<`で始まる行も削除の色にします。

`code-line-numbers="true"`を付けると、行番号を出します。`startFrom`で最初の番号を指定できます。

````
```{.python code-line-numbers="true" startFrom="11"}
def add(a, b):
    return a + b
```
````

```
def add(a, b):
    return a + b
```

コードは、ファイルを参照して載せることもできます。`include`に原稿からの相対パスを書き、ブロックの中は空にします。ファイル名は左上に出ます。

````
```{.python include="src/app.py"}
```
````

- 読み込めるのは、原稿のフォルダの中のファイルだけです。外のファイル、URL、ネットワーク上の場所、2MB を超えるファイル、文字でないファイルは読み込まず、警告を出します。
- 左上に別の名前を出すときは、`filename="app.py"`も書きます。

コードの大きさによって、そのまま表示するか、文字を縮めて 1 ページに収めるか、付録へ移すかを自動で決めます(設定の`code-display`)。下限まで縮めても 1 ページに入らないコードは、文書の最後の「付録 A コード」の章へ移し、本文にはその節へのリンクを出します。コードごとに決めるときは、`.show`(そのまま)、`.fit`(縮める)、`.appendix`(付録へ移して本文に載せない)、`.attach`(PDF の添付ファイルにして本文に載せない)、`.auto`(自動)を書きます。

````
```{.python .appendix include="src/batch.py"}
```
````

付録へ移したコードの場所には、「このコード(10 行)は、付録の A.1 に載せています。」の文を出します。文は設定の`code-appendix-note`で書き換えられ、コードごとには`note`で上書きできます。`{ref}`は付録の節の番号(リンク)、`{name}`はファイル名、`{lines}`は行数に置き換わります。

````
```{.python .appendix include="src/batch.py" note="夜間の処理は {ref} の {name} です({lines} 行)。"}
```
````

`.attach`で添付ファイルにしたコードの場所には、「このコード(10 行)は、添付ファイル batch.py にあります。」の文を出します(設定の`code-attach-note`)。表示したコードも添付するには、設定の`code-attach`を使います。添付ファイルのある PDF は会社によっては受け取れないので、既定では添付しません。

すべてのコードに行番号を付けるには、設定の`code-line-numbers: true`を使います。このとき、特定のコードだけ外すには`code-line-numbers="false"`を付けます。

コードに題と番号を付けて参照するときは、`{#lst-名前}` の囲みに入れます。

````
::: {#lst-sample}
```python
print("hello")
```

コードの題
:::

@lst-sample を参照してください。
````

コード 6-1: コードの題

```python
print("hello")
```

コード 6-1 を参照してください。

6.11 数式

文中の数式は`$E = mc^2$`のように書くと、$E = mc^2$ と組まれます。日本語に直接続けても、自動で少し間を空けます。独立した数式は`$$` で囲みます。独立した数式には、自動で番号が入ります。 `{#eq-名前}` を付けると、 `@eq-名前` で参照できます。

```
$$
\int_0^1 x^2 \, dx = \frac{1}{3}
$$ {#eq-sample}
```

$$\int_0^1 x^2 \, dx = \frac{1}{3} \tag{6-1}$$

式 (6-1) のように参照できます。番号を入れない式には`\notag` か `\nonumber` を書きます。

```
$$
a^2 + b^2 = c^2 \notag
$$
```

$$a^2 + b^2 = c^2$$

ラベルのある式だけに番号を入れるときは、設定の`equation-numbering: labeled`を使います。

`\tag{...}`を使うと、番号を手で指定できます。この番号は`@eq-名前` では参照できず、`{#eq-名前}`を付けても無視します。自動の番号も進みません。

```
$$
\sum_{i=1}^{n} x_i = \frac{1}{n} \left( \sum_{i=1}^{n} y_i \right)^2 \tag{A.1}
$$
```

$$\sum_{i=1}^{n} x_i = \frac{1}{n}\left(\sum_{i=1}^{n} y_i\right)^2 \tag{A.1}$$

行列、場合分け、複数行の揃えも書けます。

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \det A = \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix} = -2, \quad f(x) = \begin{cases} x^2 & (x \geq 0) \\ -x & (x < 0) \end{cases} \tag{6-2}$$

$$\begin{aligned} (x+1)^2 &= x^2 + 2x + 1 \\ &= x(x+2) + 1 \end{aligned} \tag{6-3}$$

複数行の式には、全体で番号が 1 つ入ります。一部の行にだけ `\notag` を書いても番号は残り、全部の行に書くと消えます。

Qiita の書き方も使えます。` ```math ` のブロックは独立した数式に、`x^2` は文中の数式にします。`$$` の外に書いた `\begin{align}` から `\end{align}` までも、独立した数式にします(`equation` と `gather` なども同じです)。

数式は LaTeX の記法で書き、組版のときに変換します。変換できない命令があると、その数式は LaTeX のまま出て、警告が出ます。数式のフォントは設定で変更できます。

6.12 定理と証明

定理や定義には、`{#thm-名前}` のように種類を表す接頭辞の囲みを使います。番号が自動で入り、`@thm-名前` で参照できます。囲みの先頭の見出しは、定理の名前になります。

```
::: {#thm-pythagoras}
## 三平方の定理
直角三角形では $a^2 + b^2 = c^2$ です。
:::

@thm-pythagoras を参照してください。

::: {.proof}
証明の本文です。
:::
```

定理 6-1 (三平方の定理). 直角三角形では $a^2 + b^2 = c^2$ です。

定理 6-1 を参照してください。

証明. 証明の本文です。

表 6-9: 定理の種類

接頭辞	種類
`thm-`	定理
`lem-`	補題
`cor-`	系
`prp-`	命題
`def-`	定義
`exm-`	例
`exr-`	練習

6.13 囲み(Qiita と Zenn の記法)

Qiita と Zenn の `:::` の記法で、色の付いた囲みを書けます。

```
:::message
Zenn の記法です。:::note info などの Qiita の記法も書けます。
:::

:::details 詳しい説明の題
題を付けた囲みです。
:::
```

`:::message` は黄色の囲みです。

`:::info` と `:::note info` は青の囲みです。 `:::note` だけでも同じです。

`:::note warn` と `:::warning` はオレンジの囲みです。

`:::message alert` 、 `:::note alert` 、 `:::alert` は赤の囲みです。

`:::tip` は緑の囲みです。

詳しい説明の題

`:::details 題` は灰色の囲みです。PDF では折りたたまず、中身を常に表示します。

`:::info 題` のように種類の後に文字を書くと、題になります。題には `` `コード` `` や `**太字**` も書けます。囲みの中にリスト、コード、別の囲みも書けます。別の囲みを入れるときは、外側のコロンの数を増やします。HTML の `<details><summary>題</summary>` も、 `:::details 題` と同じ囲みになります。

6.14 記入の指示の囲み

テンプレートの書き方の説明は、`::: {.guide}` の囲みに書きます。青い破線の囲みで出ます。テンプレートから書いた文書を仕上げるときは、設定の `guides: false` で組むと、記入の指示をまとめて消せます。

```
::: {.guide}
＊ 本書の目的と、対象の範囲を書く。
:::
```

＊ 本書の目的と、対象の範囲を書く。

6.15 注意書きの囲み(callout)

題の付いた枠には、Quarto の callout を使います。種類は `note`、`warning`、`important`、`tip`、`caution` の 5 つです。

```
::: {.callout-warning}
保存してから実行してください。
:::

::: {.callout-tip title="タイトルも付けられます"}
既定の「ヒント」の代わりに、この題が出ます。
:::
```

警告

保存してから実行してください。

タイトルも付けられます

既定の「ヒント」の代わりに、この題が出ます。

callout は Quarto の記法です。GitHub のプレビューでは枠にならず、文字のまま見えます。

6.16 Mermaid の図

GitHub と同じ書き方で図を描けます。図は本文の幅とページの高さに収まるように縮小します。小さい図は拡大しません。

````
```mermaid
flowchart LR
    A[入力] --> B[処理] --> C[出力]
```
````



キャプションと番号を付けるときは、図の先頭に`%%|`の行を足します。`fig-cap`だけでも番号が付きます。`label`も書くと`@fig-名前`で参照できます。

```
```mermaid
%%| label: fig-sample
%%| fig-cap: 図のキャプション
flowchart LR
    A --> B
```
```

図は元の大きさで描き、本文の幅に収まるように縮めます。幅を決めるときは、先頭の`%%|`の行に`width: 300`(px)か`width: 50%`(本文の幅に対する割合)を書きます。本文の幅を超える値は本文の幅にします。使えない値は警告を出して無視します。図は中央に置きます。左や右に寄せるときは`%%| fig-align: left`のように書きます。題も一緒に寄ります。

```
```mermaid
%%| width: 50%
flowchart LR
    A --> B
```
```

図の中に Mermaid の設定(`%%{init: ...}%%`の行)を書いたときは、Pubpdf の設定に重ねて、書いた値を使います。

図の色と線は、Pubpdf が描く図(節 6.17)にそろえます。

- 箱は淡い青で、枠は灰色です。判断の菱形は淡い黄色、`subgraph`の枠は淡い灰色にします。
- 文字は本文と同じ日本語のフォントで、線の文字は白地に置きます。
- flowchart の線は、角を少し丸めた折れ線にします。

線を Mermaid の既定の曲線に戻すときは`%%{init: {"flowchart": {"curve": "basis"}}}%%`と書きます。木の形の画面遷移図のように、線を縦と横だけで引きたいときは`"stepAfter"`が向いています。ただし線どうしが重なりやすくなります。`style`や`classDef`で付けた色は、そのまま使います。

Mermaid の見た目のままにするときは、`%%{init: {"theme": "default"}}%%`のように theme を選びます。theme を選んだ図には、Pubpdf の色とフォントを付けません。図の先頭の`---`の中に`theme:`を書いたときも同じです。

`subgraph`の題は折り返さず、1 行で描きます。題が枠の中の図形の並びより長いと、枠が題に合わせて広がり、隣の枠や図形と重なることがあります。そのときは題を短くしてください。題を`<br>`で 2 行にすると、2 行目が枠の中の図形と重なります。

6.17 Mermaid にない図

コードブロックの言語を次の名前にすると、図になります。Typst が本文と同じフォントで描くので、文字を検索でき、ブラウザも要りません。

表 6-10: 図の種類

言語	図
`tree`	ディレクトリ構造
`chart`	棒、折れ線、円グラフ
`schedule`	工程表
`bits`	ビット配置
`layers`	層構造
`timing`	信号の波形
`wbs`	作業分解、組織図
`wireframe`	画面の模式図
`data`	JSON と YAML の図
`network`	ネットワーク図
`usecase`	ユースケース図
`swimlane`	業務フロー(スイムレーン)
`fta`	故障の木(FTA)
`grid`	帳票と画面の方眼
`er`	ER 図
`screenflow`	画面遷移図

どの図も、先頭の行で番号と題を付けられます。書き方は Mermaid と同じです。

```
%%| label: fig-名前
%%| fig-cap: 図の題
```

- `fig-cap` を書くと「図 1: 題」と出て、図の一覧に載ります。`label` も書くと `@fig-名前` で参照できます。
- `%%| width: 300` (px)か `%%| width: 50%` (本文の幅に対する割合)で、図の幅を決められます。図はその幅で描いて、中央に置きます。`%%| fig-align: left` (`center`、`right`)で寄せを変えられます。題も一緒に寄ります。
- 題のある図は 1 ページに収まる大きさに縮めます。題のない図はページをまたぎます。
- 読めない内容は、該当する行を警告で示して、コードブロックのまま出します。

6.17.1 ディレクトリ構造

字下げか、`tree` コマンドの出力をそのまま書きます。`#` の前後に空白を入れて書いた文字は、右の列の説明になります。末尾が `/` の名前と、下に項目のある名前は太字になります。

````
```tree
project/
  src/              # ソースコード
    main.py         # 入口
  README.md
```
````

project/
├ **src/** ソースコード
│ └ main.py 入口
└ README.md

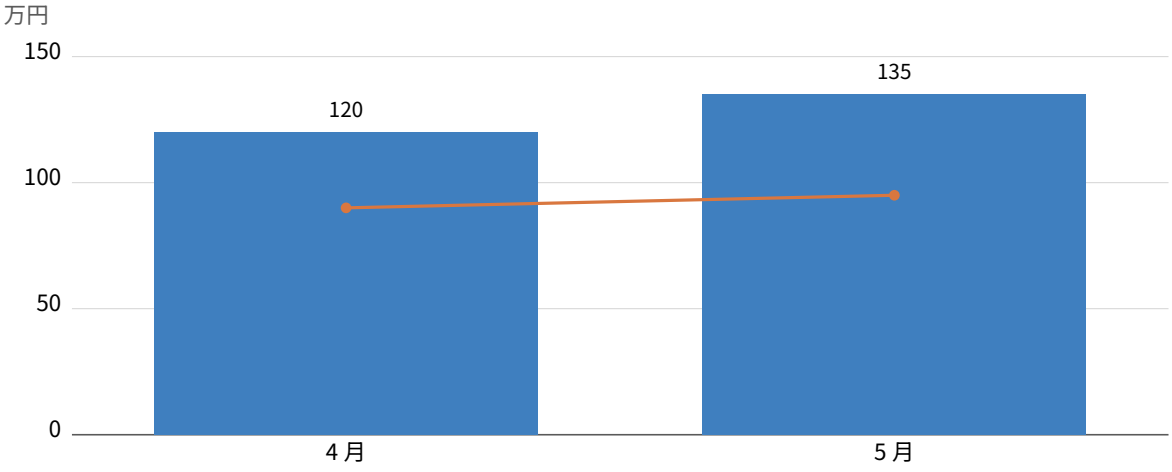
6.17.2 グラフ

Markdown の表から作ります。1 列目が項目で、2 列目からが値です。先頭の行で種類を指定します。

表 6-11: グラフの指定

キー	内容
`type`	`bar` (棒、既定)、 `line` (折れ線)、 `pie` (円)。円は 2 列目だけを使います
`lines`	棒グラフの中で折れ線にする列の名前
`unit`	単位

````
```chart
unit: 万円
lines: 費用
| 月 | 売上 | 費用 |
|---|---|---|
| 4 月 | 120 | 90 |
| 5 月 | 135 | 95 |
```
````





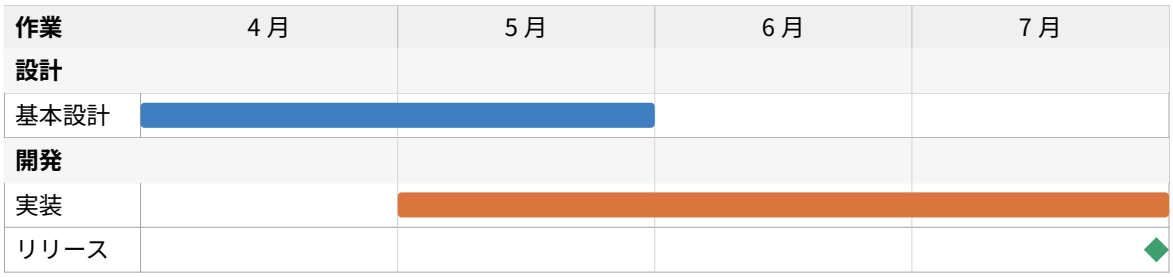
6.17.3 工程表

「作業の名前: 期間」の行を並べます。`:` のない行はグループの見出しです。名前の先頭に `◆` を書くと、期間の終わりに節目の印を出します。組んだ日の線は入らないので、同じ原稿からは同じ図ができます。

期間の書き方は 2 通りです。

- `columns: 4月, 5月, 6月` で列を指定して、`4月-5月` や `6月` と書く
- `2026-04-01 ~ 2026-05-15` や `2026-04 ~ 2026-06` と日付で書く。列は月ごと、3 年を超えると年ごとになります

```
```schedule
columns: 4月, 5月, 6月, 7月
設計
基本設計: 4月-5月
開発
実装: 5月-7月
◆ リリース: 7月
```
```



6.17.4 ビット配置

「ビット数 名前」の行を左から順に並べます。`0-3: 名前` のように番号の範囲でも書けます(混ぜられません)。名前のない項目や名前が `-` の項目は、色のない予約の欄になります。`width` は 1 行のビット数で、既定は 16 です。`msb: true` にすると、左を大きい番号にします。

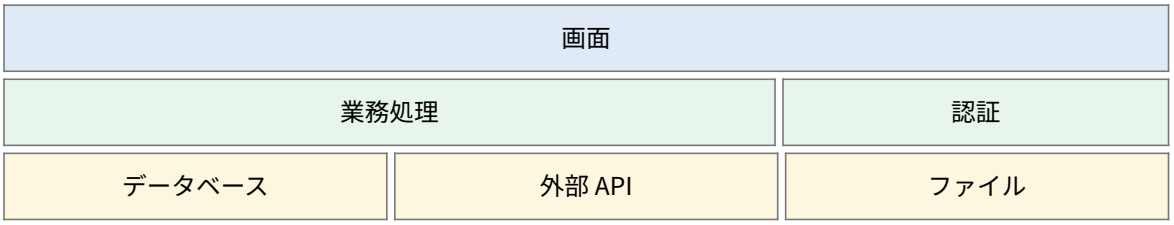
```
```bits
width: 16
4 版
4 長さ
8 種別
16 チェックサム
```
```



6.17.5 層構造

1 行が 1 層です。`|` で区切ると横に並びます。`*2` のように書くと、その幅の割合になります。

````
```layers
画面
業務処理 *2 | 認証
データベース | 外部 API | ファイル
```
````



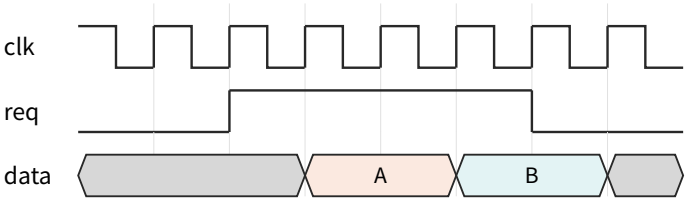
6.17.6 信号の波形

「信号の名前: 波形」の行を並べます。波形の 1 文字が 1 区間です。`=` の値は、波形の後ろに `,` で区切って書きます。

表 6-12: 波形の文字

文字	意味
`0`、`1`	低、高
`p`、`n`	クロック。`p` は立ち上がりから、`n` は立ち下がりから
`x`	不定
`z`	ハイインピーダンス
`=`	値のある区間
`.`	前の続き。クロックでは次の周期

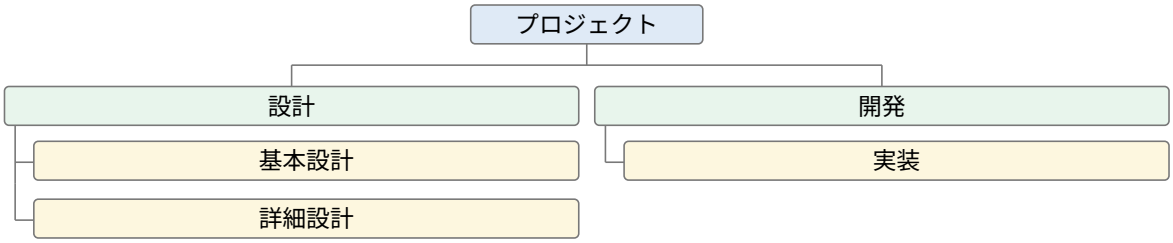
````
```timing
clk: p.......
req: 0.1...0.
data: x..=.=.x A, B
```
````



6.17.7 作業分解と組織図

字下げで階層を書きます。1 段目が横に並び、その下は字下げして縦に並びます。一番上の項目が 1 つなら、それを全体の上に置きます。

```
```wbs
プロジェクト
  設計
    基本設計
    詳細設計
  開発
    実装
```
```



6.17.8 画面の模式図

1 行が 1 段で、`|` で区切ると横に並びます。

表 6-13: 画面の部品

書き方	部品
最初の行の `# 題`	画面の題の帯
2 行目以降の `# 文字`	太字の見出し
`[文字]`	ボタン
`"文字"`	入力欄
`^文字^`	選択欄
`[ ] 文字`、`[x] 文字`	チェックボックス
`( ) 文字`、`(x) 文字`	ラジオボタン
`--`	区切り線

```
```wireframe
# ログイン
利用者名 | "名前を入力"
区分 | ^一般^
---
[x] ログインを保持
[ログイン] | [取り消し]
```
```



区分 一般

☒ ログインを保持

ログイン 取り消し

6.17.9 JSON と YAML の図

JSON か YAML をそのまま書きます。入れ子は箱の中の箱になります。

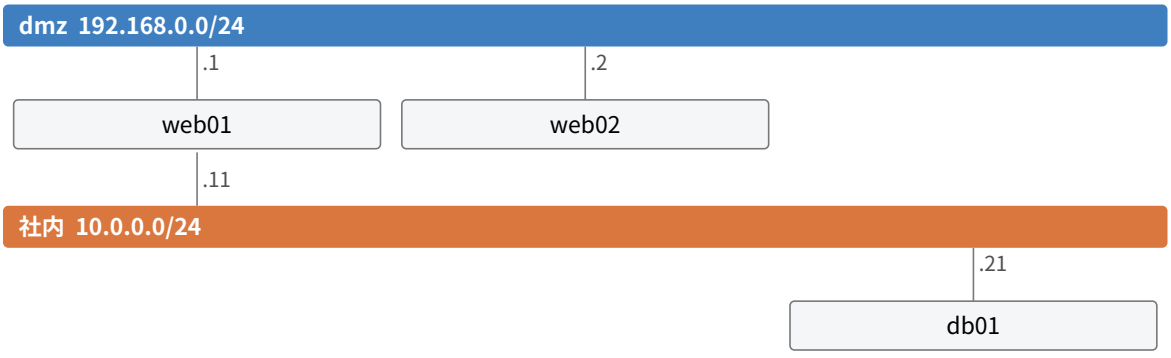
```data
name: 製品 A
tags: [新規, 重要]
owner:
  team: 開発
  members: 3
```

name	製品 A
tags	0 新規 1 重要
owner	**team** 開発 **members** 3

6.17.10 ネットワーク図

字下げしない行がネットワークで、字下げした行がそのネットワークにつながる機器です。行の最後の`10.0.0.0/24`や`.1`はアドレスとして読みます。同じ名前の機器は 1 つにまとめ、複数のネットワークにつなぎます。

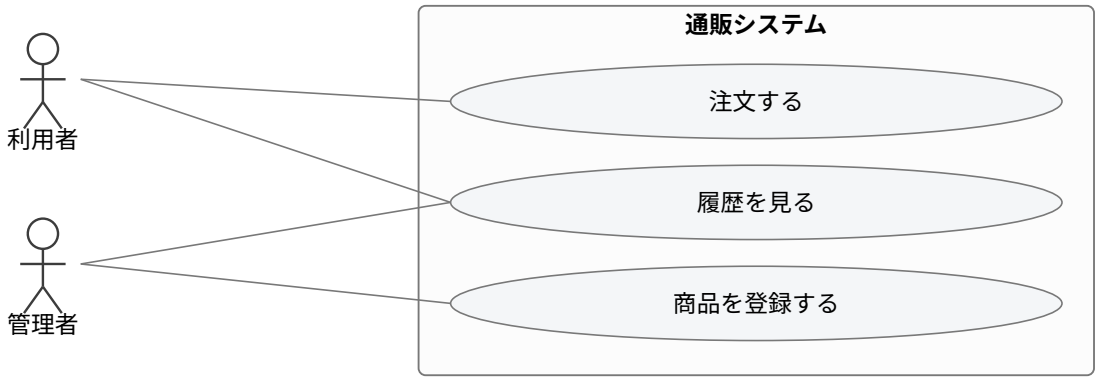
```network
network dmz 192.168.0.0/24
  web01 .1
  web02 .2
network 社内 10.0.0.0/24
  web01 .11
  db01 .21
```



6.17.11 ユースケース図

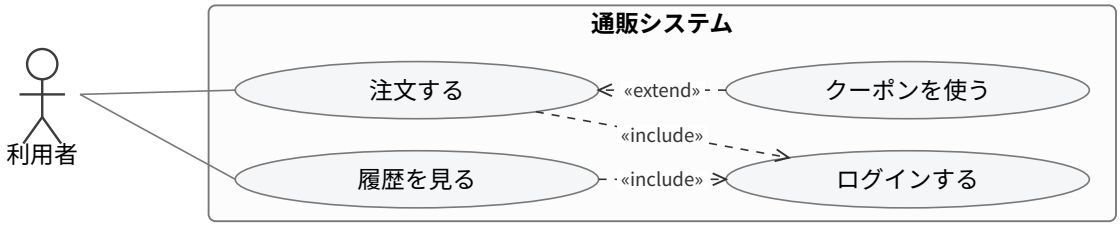
「利用者: 機能, 機能」の行を並べます。機能は`,`か`、`で区切るので、機能の名前に`、`を使うときは`・`などに置き換えてください。左に利用者、右に機能を並べて、直線で結びます。同じ名前の機能は 1 つにまとめます。`system`で枠の題を書けます。線の配置を工夫しないので、線が交差することがあります。

```
```usecase
system: 通販システム
利用者: 注文する, 履歴を見る
管理者: 商品を登録する, 履歴を見る
```
```



ユースケースどうしの関係は、`include: 機能 -> 機能`と`extend: 機能 -> 機能`の行で書きます。破線の矢印と «include»、«extend» の文字で結びます。矢印は、include では含む機能から含まれる機能へ、extend では拡張する機能から拡張される機能へ向けて書きます。`,`で区切ると、まとめて書けます。利用者とつながる機能を左の列に、関係だけでつながる機能を右の列に置きます(3 列まで)。

```
```usecase
system: 通販システム
利用者: 注文する, 履歴を見る
include: 注文する, 履歴を見る -> ログインする
extend: クーポンを使う -> 注文する
```
```



6.17.12 業務フロー(スイムレーン)

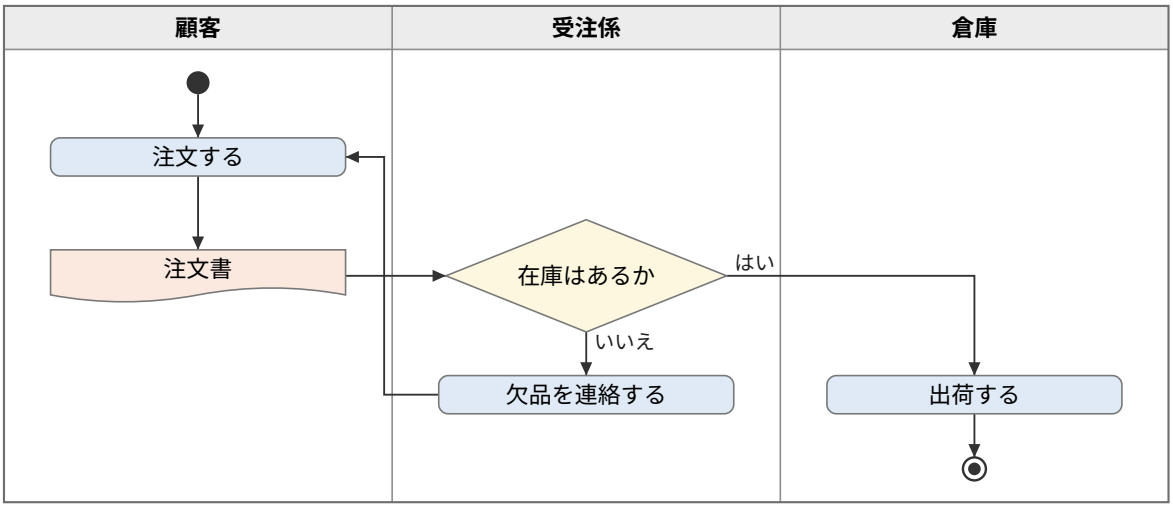
「レーン: 記号」の行を、処理の順に上から書きます。レーンが列になり、記号はつながる元の記号より下の段に置きます。ほかのレーンへ渡す記号は、間のレーンが空いていれば同じ段に並べて横の矢印でつなぎ、つながりのない記号どうし(「はい」と「いいえ」の先など)も、レーンが違えば同じ段に並べます。

表 6-14: スイムレーンの記号

書き方	記号
`文字`	処理(角丸の箱)
`◇ 文字`	判断(菱形)
`●`、`● 文字`	開始
`◎`、`◎ 文字`	終了
`[書類] 文字`	帳票、書類
`[データ] 文字`	データの保管(円柱)

- 記号は、次の行の記号へ矢印でつながります。終了からはつながりません。
- `-> 行き先`を書くと、次の行ではなく行き先へつなぎます。行き先は、ほかの行の文字と同じに書きます。`,`で区切ると複数につなぎ、`はい: 行き先`のように書くと線に文字を付けます。前の行の処理を書くと、戻る矢印になります。`->`の代わりに`→`も使えます。
- `->`だけを書くと、どこへもつなぎません。
- 同じ文字の記号が複数あるときは、その行より後で最も近いものにつなぎます。
- `lanes`でレーンの並びを書けます。書かないと、出てきた順に並びます。
- 判断の文字が長くレーンが細いときは、菱形をレーンの幅まで広げ、文字を少し小さくします。「はい」「いいえ」が菱形の横に入らなければ、隣のレーンへ出た線の上か、菱形の斜めの辺の上に置きます。

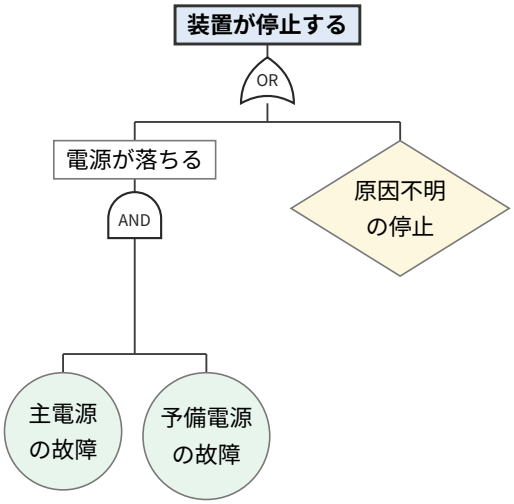
```
```swimlane
lanes: 顧客, 受注係, 倉庫
顧客: ●
顧客: 注文する
顧客: [書類] 注文書
受注係: ◇ 在庫はあるか -> はい: 出荷する, いいえ: 欠品を連絡する
倉庫: 出荷する
倉庫: ◎
受注係: 欠品を連絡する -> 注文する
```
```



6.17.13 故障の木(FTA)

字下げで木を書きます。一番上の行が頂上の事象です。下に事象のある事象は長方形になり、先頭に`AND`か`OR`を書くと、その下にゲートを描きます(下に事象のある事象には、どちらかが要ります)。下に事象のない事象は基本の事象で、円になります。先頭に`◇`を書くと、それ以上展開しない事象(菱形)になります。

```
```fta
OR 装置が停止する
  AND 電源が落ちる
    主電源の故障
    予備電源の故障
  ◇ 原因不明の停止
```
```



事象の名前の中の`<br>`は改行になります。`AND`、`OR`、`◇`で始まる名前を文字のまま書くときは、`\OR 回路の誤配線`のように先頭に`\`を付けます。

基本の事象が多くて横に並べると幅に入らないときは、基本の事象だけを下に持つゲートの下に、事象を縦に並べます(円や菱形の右に名前の箱を置きます)。縦に並べられない形の木は、事象の名前を箱に入れ、その下に小さな円や菱形を付けた形にして幅を詰めます。

6.17.14 帳票と画面の方眼

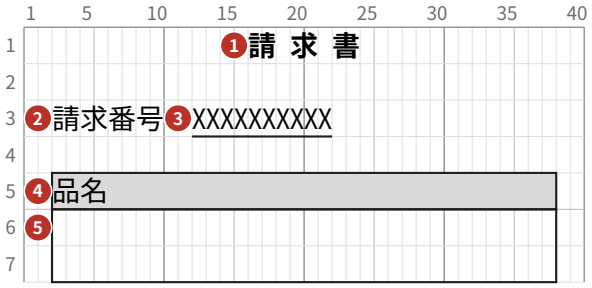
「行,桁:文字」の行で、方眼の上に項目を置きます。行と桁は 1 から数え、全角の文字は 2 桁、半角の文字は 1 桁です。`3, 12-21`のように桁の範囲を書くと、その範囲に入れ、入らない文字は次の行へ折り返します。行の範囲は`5-8, 2-39`のように書きます。桁の範囲を書かないと、文字の幅だけの範囲になります。

表 6-15: 方眼の書き方

書き方	内容
`columns`、`rows`	方眼の桁と行の数。書かないと項目の範囲から決めます
先頭の`①`、`(12)`	項目番号。項目の左か上の空いた桁に置きます
末尾の`{枠}`	枠線
`{網掛け}`	網掛け
`{下線}`	下線
`{太字}`	太字
`{右}`、`{中央}`	右寄せ、中央寄せ。中央は上下にも寄せます

`{枠 網掛け}`のように並べて書けます。文字のない枠は`6-7, 3-38: {枠}`のように範囲を書きます。項目の一覧(番号、名前、桁数など)は、表で本文に書いてください。

````
```grid
columns: 40
rows: 7
① 1, 17: 請 求 書 {太字}
② 3, 3: 請求番号
③ 3, 13-22: XXXXXXXXXX {下線}
④ 5, 3-38: 品名 {網掛け 枠}
⑤ 6-7, 3-38: {枠}
```
````



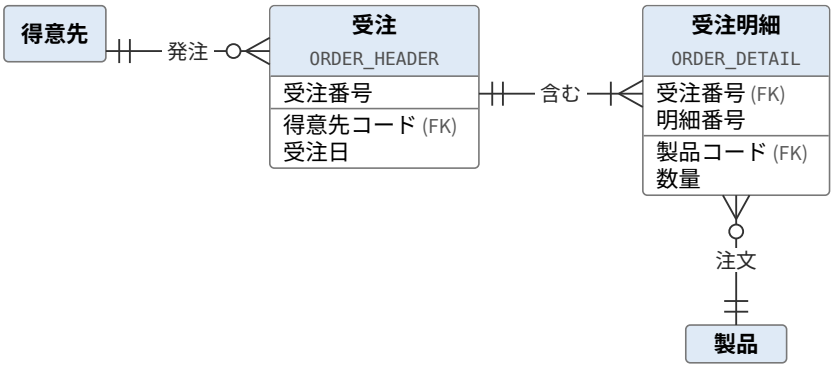
6.17.15 ER 図

Mermaid の`erDiagram`と同じ書き方で、関係の行と、属性を並べる`{ }`を書きます。言語の名前を`mermaid`から`er`に変えるだけでも描けます(`erDiagram`と`direction`の行は読み飛ばします)。箱は IE 記法で、名前の下に主キーの属性を、線で区切ってほかの属性を並べます。

- 関係は`A ||--o{ B : 関係の名前`の形で書きます。記号の左が A の側、右が B の側の多重度で、`||`は 1、`|o`と`o|`は 0 か 1、`}|`と`|{`は 1 以上、`}o`と`o{`は 0 以上です。線の端に、縦線、丸、鳥の足で描きます。`--`は実線(依存する関係)、`..`は破線(依存しない関係)です。関係の名前が`""`なら、線に名前を付けません。
- 属性は`A {`と`}`の間に、1 行に 1 つ書きます。後ろに`PK`、`FK`、`UK`を付けられ、`PK, FK`のように並べられます。`PK`の属性は区切りの線の上に並び、`FK`と`UK`は名前の後ろに小さく出ます。名前の前に型を書くと(`string 得意先コード`)、右に小さく出ます。後ろの`"説明"`は図に描きません。
- `受注 ORDER_HEADER {`のように、エンティティ名の後に空白で区切ってテーブル名を書くと、名前の下に出ます(Mermaid にはない書き方です)。

- `|` で始まり `|` で終わる行を並べると、箱をその段と列に置きます。空のセルの所はあけておきます。置き場所に書かなかった箱は、つながる箱の近くに置きます。置き場所の行がなければ、多重度が「1」の側の箱を上の段に置きます(`direction LR` なら左の列)。
- 線は、箱の間の溝を縦と横に通ります。隣どうしの箱はまっすぐ結ぶので、関係のある箱を隣に置くと読みやすくなります。

````
```er
| 得意先 | 受注 | 受注明細 |
|        |      | 製品     |
得意先 ||--o{ 受注 : 発注
受注 ||--|{ 受注明細 : 含む
製品 ||--o{ 受注明細 : 注文
受注 ORDER_HEADER {
  受注番号 PK
  得意先コード FK
  受注日
}
受注明細 ORDER_DETAIL {
  受注番号 PK, FK
  明細番号 PK
  製品コード FK
  数量
}
```
````



6.17.16 画面遷移図

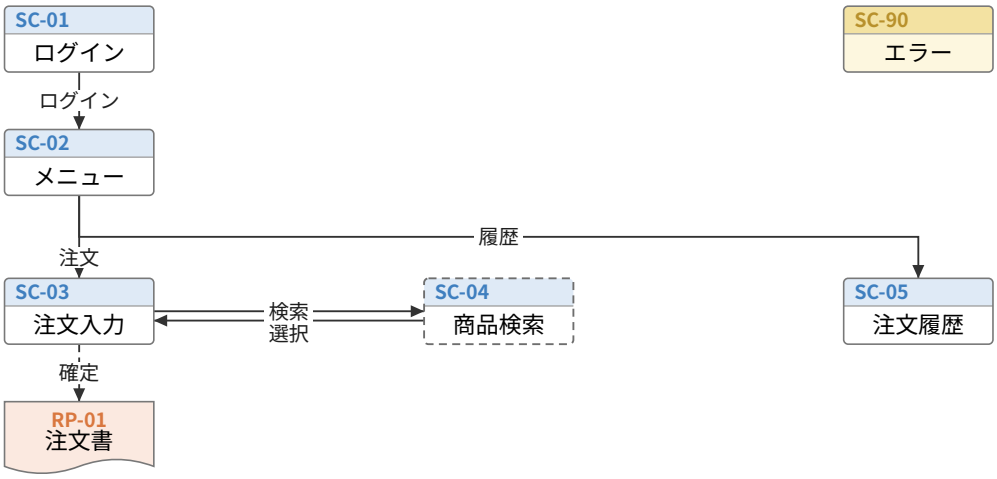
「画面 -> 操作: 画面」の行で遷移を書きます。画面は「ID 名前」の形で書き、最初の空白までを ID とみなします。名前はどこか 1 か所に書けばよく、2 回目からは ID だけで書けます。画面は窓の形で描き、上の帯に ID を、下に名前を出します。

表 6-16: 画面遷移図の書き方

書き方	内容
`A -> 操作: B`	A から B への遷移。矢印に操作を付けます。`操作:`を省くと、文字のない矢印になります
`A -> B, C`	複数の画面へつなぎます。`A, B -> C`のように元も並べられます
`A ..> B`	破線の矢印
`画面: ID 名前`	画面。書かなくても、出てきた ID は画面になります
`小画面: ID 名前`	別ウィンドウの小画面。破線の枠で描きます
`帳票: ID 名前`	帳票。文書の形で描き、帳票への矢印は破線にします
`共通: ID 名前`	どの画面からも移る画面(エラーなど)。黄色で描きます
`外部: ID 名前`	ほかのシステム。灰色の箱で描きます

- `,`で区切ると、まとめて書けます。名前に`、`を使えるよう、`、`では区切りません。
- `|`で始まり`|`で終わる行を並べると、画面をその段と列に置きます(ER 図と同じ書き方です)。空のセルの所はあけておきます。置き場所に書かなかった画面は、つながる画面の隣に置きます。
- 置き場所の行がなければ、矢印の入ってこない画面を一番上の段にし、遷移をたどった深さの段に並べます(1 段に 5 つまで)。
- 線は、画面の間の通り道を縦と横に通ります。同じ画面から出る線は根元を 1 本にまとめて枝分かれさせ、行き来する 2 本の線は並べて引きます。
- 同じ 2 つの画面の間に操作を 2 回書くと、1 本の矢印に`、`で並べます。

````
```screenflow
| SC-01 ログイン |                | SC-90 エラー    |
| SC-02 メニュー |                |                 |
| SC-03 注文入力 | SC-04 商品検索 | SC-05 注文履歴 |
| RP-01 注文書   |                |                 |
小画面: SC-04
帳票: RP-01
共通: SC-90
SC-01 -> ログイン: SC-02
SC-02 -> 注文: SC-03, 履歴: SC-05
SC-03 -> 検索: SC-04
SC-04 -> 選択: SC-03
SC-03 -> 確定: RP-01
```
````



6.18 画像

画像は、原稿からの相対パスで指定します。`{#fig-名前}`を付けると本文から参照でき、`width`で幅を指定できます。

```
![図のキャプション](img/photo.png){#fig-photo width=60%}
```

キャプションか`{#fig-名前}`があれば番号が付きます。どちらもない画像には番号が付きません。

Zenn の書き方の`![](img/photo.png =300x)`でも幅を指定できます。数は px(96 分の 1 インチ)で、`=50%x`は本文の幅に対する割合です。本文の幅を超える値は、本文の幅にします。高さの指定(`=x200`)は使わず、警告を出します。Zenn と同じく、画像だけの行のすぐ次の行に`*題*`と書くと、図の題になります。

```
![設定パネルの画面](img/photo.png =300x)
*設定パネル*
```

HTML の`<img src="img/photo.png" width="300">`も画像として出します。`width`の数は px です。

`https://`などの URL と、`//サーバ/共有`のようなネットワーク上の画像は読み込みません。受け取った原稿を組むだけで外部へ通信しないようにするためです。画像の場所には、読み込まなかったことを示す灰色の枠が入り、キャプションと番号は残ります。画像を使うときは、原稿のフォルダに保存して相対パスで書いてください。

画像だけを置いた段落の画像は、題の有無や幅の指定によらず中央に置きます。文の途中に置いた画像は、文字と同じ行の中に置きます。位置を変えるときは`fig-align`を`left`、`center`、`right`にします。題も一緒に寄ります。Zenn の書き方では`![説明](img/photo.png =300x){fig-align="left"}`、HTML では`<img src="img/photo.png" align="left">`と書けます。`::: {.right}`などの寄せの区画の中の画像は、区画の寄せに従います。横に並べた図には効きません。

```
![左に寄せた図](img/photo.png){width=40% fig-align="left"}
```

縦に長い図は、`.fit` を書くと、図の題と一緒に 1 ページに収まる大きさに縮めます。さらに、ページの残りに入るなら(元の大きさの半分までは)残りに合わせて縮め、前の文に続けて置きます。半分より小さくなるときは縮めず、次のページへ送ります。`.fit` を付けない画像は、ページより高いと題がページの下にはみ出すことがあります。`.fit` は画像だけを置いた段落に使ってください。

```
![システムの構成](img/architecture.png){#fig-architecture .fit}
```

画像の幅(`=300x`)は、題(`"..."`)の前にも後ろにも書けます。

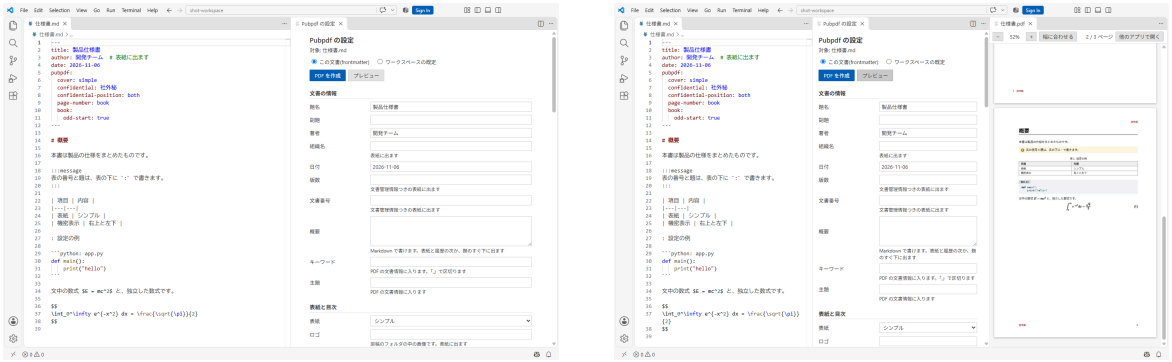
図を横に並べるときは、`layout-ncol` に列の数を書いた囲みに入れます。囲みの最後の段落が、全体の題になります。全体と個々の図の両方を参照できます。

```
::: {#fig-pair layout-ncol=2}
![設定パネル](img/vscode-panel.png){#fig-pair-panel}

![プレビュー](img/vscode-preview.png){#fig-pair-preview}

VS Code の画面
:::

@fig-pair の @fig-pair-panel が設定パネルです。
```



(a) 設定パネル　　(b) プレビュー

図 6-1: VS Code の画面

図 6-1 の 図 6-1a が設定パネルです。

6.19 改ページと横向きのページ

改ページしたい場所に、次のいずれかを 1 行で書きます。

```
{{< pagebreak >}}

\newpage
```

`\pagebreak` と `\clearpage` も同じです。文の途中やコードブロックの中では改ページしません。続けて書いても白紙にはなりません。章は新しいページから始まるので、章のファイルの

先頭に書いても何も起きません。引用、囲み、リストの中では改ページできないので、警告を出して無視します。

幅の広い表などは、`.landscape` の囲みに入れると、その部分だけ横向きのページになります。囲みの前後で改ページします。囲みの直前の見出しは、横向きのページへ移します(縦のページに見出しだけが残らないように)。囲みの中の表は、横向きのページの幅で列を配分します。引用、囲み、リストの中では、警告を出して縦のまま組みます。文書全体を横向きにする設定は `orientation` です。

```
::: {.landscape}
横向きのページに置く内容です。
:::
```

6.20 Typst を直接書く

設定や Markdown の記法で足りないときは、Typst の記法を直接書けます。

````
```{=typst}
#text(fill: red)[この文字は赤になります。]
```
````

この文字は赤になります。 Typst の記法の誤りがあると、組版がエラーで止まります。

`#image("図.png")` のように直接書いた画像のファイルは、中間フォルダへ複製されません。Markdown の本文からも `![説明](図.png)` で参照してください。

6.21 文献の引用

文献は BibTeX の形式(`.bib`)のファイルに書きます。設定の `bibliography` にファイルを指定し、本文に `[@キー]` と書くと引用になります。

```
@book{knuth1984,
  title = {The TeXbook},
  author = {Knuth, Donald E.},
  year = {1984},
  publisher = {Addison-Wesley},
}
```

```
組版の教科書 [@knuth1984] によると、次のとおりです。

ページを示します [@knuth1984, p. 10]。複数も書けます [@knuth1984; @yamada2020]。
```

- 文献の一覧は、見出し「参考文献」を付けて、文書の最後に自動で出ます。載るのは引用した文献だけです。
- 合本では、どの章からでも引用できます。文献のファイルは親 md に指定します。
- ファイルは原稿のフォルダの中の `.bib` だけを指定できます。受け取った原稿が他のファイルを読まないようにするためです。

- ファイルにない文献を `[@キー]` と書くと、文字のまま出して警告します。`[@a; @b]` の片方だけがないときは、ある方は引用になります。
- 引用が 1 つもなければ、文献の一覧は出ません。
- エントリは `@book{...}` のように波括弧で書きます。`@book(...)` の丸括弧は使えません。
- `fig-`、`tbl-`、`eq-`、`sec-` などで始まるキーは、図や表への参照と区別できないので使わないでください。
- ページの出し方は書式によって違います。`[@knuth1984, p. 10]` のページは、IEEE では出ますが、SIST 02 では出ません。
- 括弧のない `@knuth1984` は「著者名 (年)」の形になります。年を出さない MLA では `Knuth()` になるので、`[@knuth1984]` と書いてください。

6.22 文書の値を本文に差し込む

`{{< meta キー >}}` と書くと、frontmatter の値に置き換わります。版数や題を本文に何度も書くときに、書き直しの漏れを防げます。

```
この文書の版は {{< meta version >}} です。
```

使えるキーは `title`、`subtitle`、`author`、`date`、`version`、`doc-number` です。合本の章では親 md の値を使います。

- 値は書かれたとおりの文字で出します。Markdown としては読みません。
- コードの中では置き換えません。
- 使える場所は本文です。囲みの題(`:::info 題`)の中では置き換えません。
- 他のキーを書くと文字のまま出して警告します。値が frontmatter にないときは空にして警告します。

6.23 リンク

表 6-17: リンクの書き方

書き方	リンク先
`[文字](#見出し)`	同じファイルの見出し。日本語の見出しもそのまま書けます
`[文字](02-design.md)`	別の章の先頭
`[文字](02-design.md#見出し)`	別の章の見出し
`[文字](URL)`	Web ページ
`<https://example.com/>`	Web ページ。URL をそのまま表示します

同じ題の見出しが複数あるときは、GitHub と同じく 2 つ目を `#見出し-1` で指せます。

文書に含まれない md や、見つからない見出しへのリンクは、警告を出して文字だけにします。md 以外のファイルへのリンク(`data.csv` など)はそのまま残るので、PDF を別の場所へ移すと開けなくなります。

URL を`https://...`とだけ書いた場合も、書いたとおりの文字のリンクにします。URL の中の`_`や`*`は強調の記号になりません。URL の後ろの句読点と、対にならない閉じ括弧は URL に含めません。日本語の文字が続くと、そこで URL が終わります。

6.24 見出しへの参照

見出しに`{#sec-名前}`を付けると、本文から参照できます。`numbering`が`chapter`か`decimal`なら番号で出ます(例: 節 5.1)。章は「第 1 章」、付録の章は「付録 A」と出ます。`numbering: none`や番号のない見出し(`{.unnumbered}`)では、見出しの題が出ます。

```
## 奇数ページ始まり {#sec-odd-start}

詳しくは @sec-odd-start を参照してください。
```

6.25 書くときの注意

@ で始まる語

相互参照と、文献のファイルにあるキー以外は、ただの文字として出ます。@someone のように書いても問題ありません。ただし`fig- tbl- eq- sec- lst- tip- def-`などの接頭辞で始まる語は、相互参照と見なされます。

コードセル

```{python}`のように書いても実行しません。普通のコードブロックとして載せます。

ショートコード

`{{< env USERNAME >}}`のような Quarto のショートコードは展開せず、文字のまま出します。例外は、1 行で書いた`{{< pagebreak >}}`と、`{{< meta 版数など >}}`です。受け取った原稿が環境変数を読めないようにするためで、Quarto にも動作に必要な環境変数だけを渡します。

存在しない参照

`@fig-`などの参照先がないと、本文に`?@fig-名前`と太字で出て、警告も出ます。

空行の抜け

段落のすぐ次の行に書いた見出し、リスト、表、引用は、前に空行がないと、前の段落の文字としてつながります(Zenn や Qiita とは違います)。行番号を示して警告するので、前に空行を入れてください。番号のリストは、`1.`で始まるものだけを知らせます。

`\`で始まる語

本文の`C:\Users`、`\n`、`\section`などは、書いたとおりの文字で出します。LaTeX の命令としては読みません。ただし Markdown の決まりで、`\\`は`\`1 つに、`\*`は`*`になります。パスは`{.path}`で書くと確実です(節 6.6)。
```

第 7 章　VS Code での操作

7.1　コマンド

エディタ右上のボタン、エクスプローラーの右クリック、コマンドパレットから実行します。

表 7-1: 拡張機能のコマンド

コマンド	内容	場所
`Pubpdf: PDF を作成`	PDF を作ります	ボタン、右クリック、コマンドパレット
`Pubpdf: PDF をプレビュー`	VS Code の中で PDF を表示します。原稿の md が PDF より新しければ作り直します	ボタン、右クリック、コマンドパレット
`Pubpdf: 設定パネルを開く`	設定パネルを開きます	ボタン、コマンドパレット
`Pubpdf: frontmatter の雛形を挿入`	原稿の先頭に frontmatter を入れます	コマンドパレット
`Pubpdf: 環境を確認`	Quarto、フォント、ブラウザの状態を一覧で示します	コマンドパレット
`Pubpdf: ライブプレビューを開始/停止`	編集中の内容を、保存せずに組み直して表示し続けます。表示は作る PDF と同じです	タブの右上の `...`、コマンドパレット
`Pubpdf: 文章の誤りをまとめて直す`	文章の点検で見つかった誤りのうち、自動で直せるものを直します	コマンドパレット
`Pubpdf: 表の見やすい表示をオン/オフ(試験的)`	編集中の表の表示を切り替えます	コマンドパレット

7.2　PDF を作る

- 実行中は通知に進捗を表示します。通知の「キャンセル」で止められます。
- 未保存の変更は、保存してから組みます。
- 親の md が同じワークスペースのフォルダにあれば、章の md を開いた状態で実行しても本全体を組みます。本が複数あれば、選択肢を示します。
- 実行中にもう一度実行すると、前の処理を止めて最新の内容で組み直します。
- 設定 `pubpdf.renderOnSave` をオンにすると、保存したときに PDF を作り直します。対象は `pubpdf:` のある原稿と合本の章です。
- PDF を他のアプリで開いていて上書きできないときは、原因を知らせます。
- ワークスペースのフォルダの直下に `.git` があるときは、最初に PDF ができた後に、`.pubpdf/` を `.gitignore` に追加するかを尋ねます。`pubpdf.renderOnSave` で作ったときと、`pubpdf.buildDirectory` が `global` のときは尋ねません。
- 信頼していないワークスペース(制限モード)では、拡張機能は動きません。
- Mermaid の図は、前に描いた図を再利用します(設定 `pubpdf.diagramCache`、既定はオン)。図の中身か Quarto の版が変わったときだけ描き直すので、2 回目からは速く組めま

す。再利用した図は、描き直した図と同じに組み上がります。図は中間フォルダの `cache` に保存し、使わなくなった古いものから消します。

7.3 設定パネル

Markdown のタブの右上にある歯車のボタンか、コマンドパレットの `Pubpdf: 設定パネルを開く` で開きます。ボタンが見当たらないときは、タブの右上の `...` の中を確認してください。

図 7-1 のパネルで、表紙、目次、図と表の一覧、参考文献、機密表示の文字と位置、ページ番号の位置と数え方、用紙の種類と向き、余白、章の前の改ページ、奇数ページ始まり、綴じ代、見出しと図表と式の番号、コードの行番号、文字の大きさ、行間、字下げ、フォントを変更できます。バージョン履歴と付録はパネルにはないので、frontmatter に書きます。文献のファイルは文書ごとの設定なので、パネルの「この文書」でだけ指定できます。

段組み、ヘッダとフッタの文字、透かし、テーマの色、表の体裁、コードの配色、PDF の規格、組織名、キーワード、主題も変更できます。ロゴと文献のファイルは原稿のフォルダの中を指すので、保存先に「この文書」を選んだときだけ変更できます。

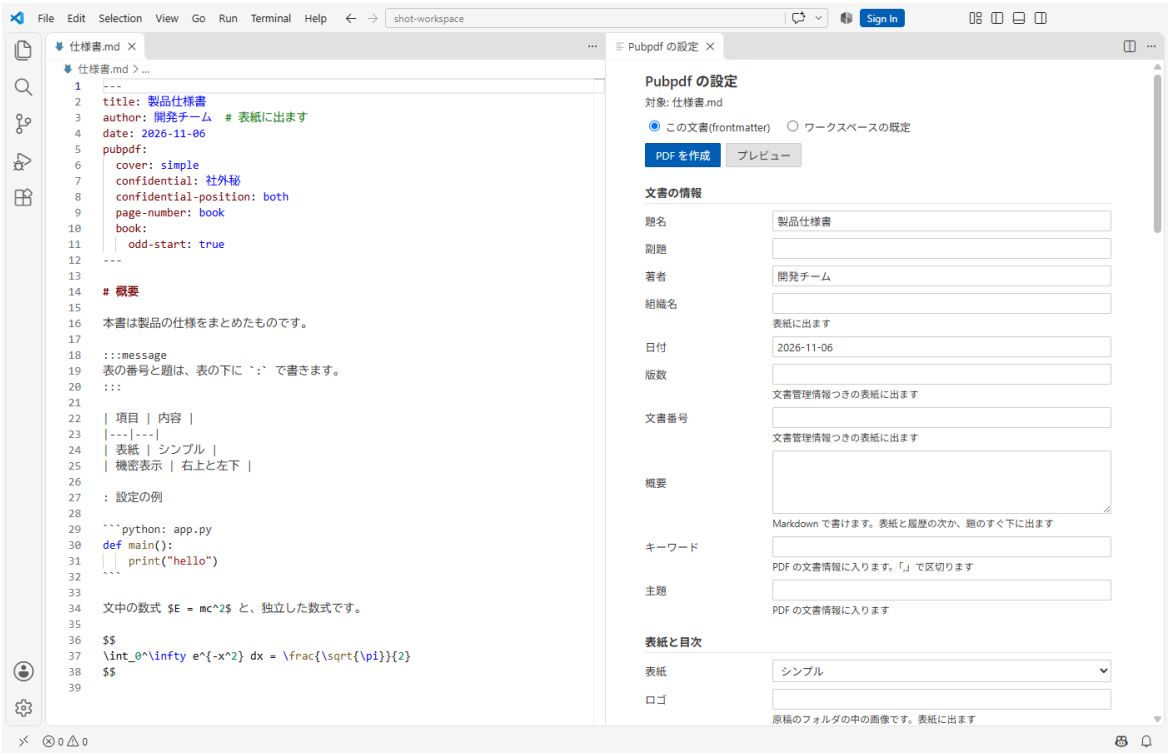


図 7-1: 設定パネル。左の原稿の frontmatter と連動します

パネルの上で保存先を選びます。

表 7-2: 設定パネルの保存先

保存先	書き込む場所	効く範囲
この文書	開いている md の frontmatter	その文書だけ
ワークスペースの既定	ワークスペースの `settings.json`	ワークスペースの文書で、frontmatter に書いていない項目

- 「既定」を選ぶと、その項目を frontmatter から消します。括弧の中に、そのときに使われる値を示します。
- frontmatter へ書き戻すときも、コメント、キーの順序、値、本文は変えません。ただし frontmatter 全体を書き直すので、値の後のコメントの前の空白、折り返した文字列の改行、リストの字下げは揃うことがあります。
- 変更は未保存の編集として入るので、取り消せます。PDF を作るときに保存します。
- 章の md を開いているときは、親の md の設定を変更します。
- フォント名の欄には、読み込めるフォントが候補として出ます。

パネルの最後の「編集中の表示(VS Code の設定)」では、表の見やすい表示、列を揃えてたたむ表示、たたむセルの長さ、行の終わりの Enter で行を足すか、ライブプレビューで編集中の箇所を追うかと待ち時間を変更できます。この欄は文書には書かず、VS Code の設定に保存します。保存先の選択によりません(ワークスペースの `settings.json` に指定があればそこを、なければユーザーの設定を書き換えます)。

7.4 プレビュー

図 7-2 のように、VS Code の中で PDF を確認できます。PDF を作り直すと、表示している位置を保ったまま更新します。拡大と縮小、幅に合わせる表示ができます。画像や VS Code の設定を変えただけでは、プレビューのコマンドは PDF を作り直しません。`Pubpdf: PDF を作成` を実行してください。

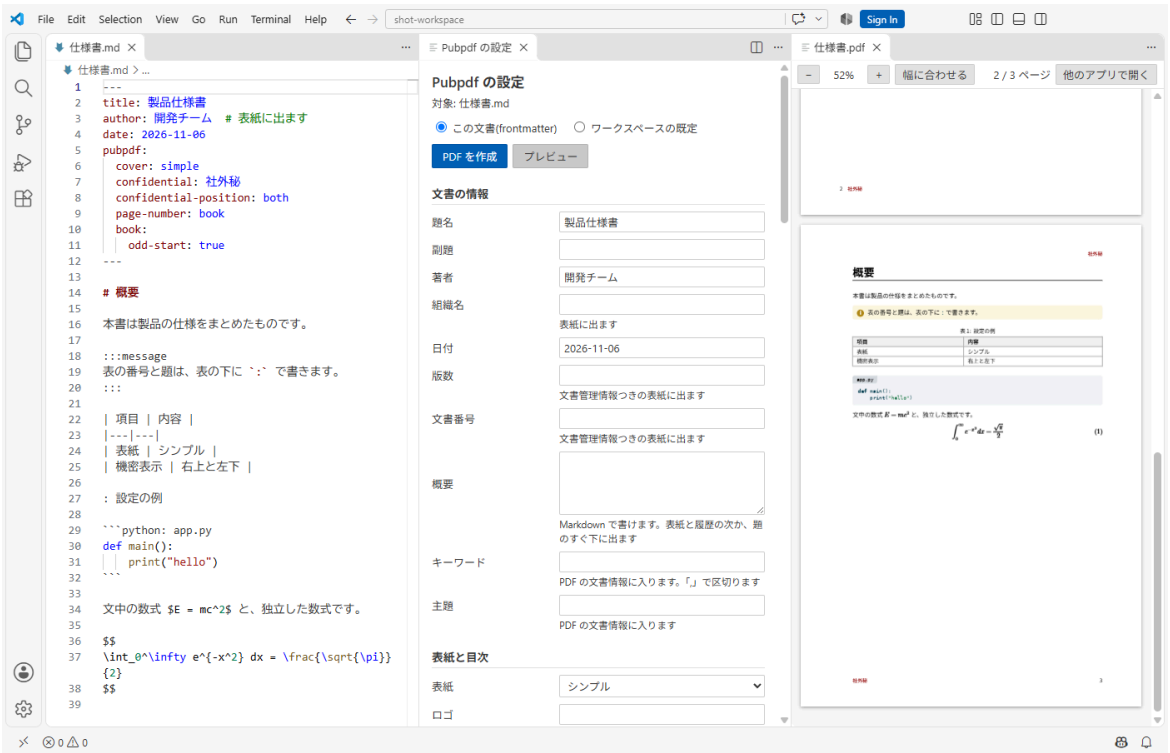


図 7-2: プレビュー。作り直しても表示位置を保ちます

7.5 ライブプレビュー

`Pubpdf: ライブプレビューを開始/停止` を実行すると、横にライブプレビューを開き(タブの題の後ろに「(ライブ)」と付きます)、保存していない内容のまま組み直して表示し続けます。表

示するのは `Pubpdf: PDF を作成` と同じ組版の PDF そのものです。用紙、余白、フォント、見出し、表、図、番号、表紙、目次の見た目は、作る PDF と変わりません。

- 編集を止めて少したったら(既定は 1 秒。設定 `pubpdf.liveDelay`)組み直します。組み始めてすぐに編集したときは組み直しをやり直し、それより後の編集は、組み終えてから続けて組みます。書き続けていても、表示は少しずつ新しくなります。
- PDF の見た目に関わる VS Code の設定(設定パネルの「ワークスペースの既定」で変えたものを含みます)を変えたときと、画像、文献のファイル、ロゴ、 `include` で読み込むコードのファイル、開いていない章の md を変えたときも組み直します。
- 組んだ PDF は中間フォルダの `live` に置きます。原稿の PDF は上書きしません。原稿の PDF は、これまでどおり `Pubpdf: PDF を作成` で作ります。
- ライブプレビュー中の文書は、保存しても(`pubpdf.renderOnSave` がオンでも)組みません。
- もう一度実行するか、ステータスバーの「Pubpdf ライブ」を選ぶか、プレビューを閉じると止まります。組んでいる間は、ステータスバーの印が回ります。
- 組み直しには、ふつうに PDF を作るのと同じ時間がかかります(数秒から十数秒)。変わったところだけを組み直すことはできません。ページの割り付け、番号、目次、参照は文書の全体で決まるためです。Mermaid の図は前に描いたものを再利用するので、図の多い文書でも 2 回目からは速くなります。

ツールバーに、表示が原稿に追いついているかを出します。

表 7-3: ライブプレビューの状態

表示	意味
最新	今の原稿と設定で組んだ PDF を表示しています
未反映の編集があります	編集を止めるのを待っています。少したつと組み直します
組み直しています…	組んでいます。表示の上端に細い帯が流れます
組めませんでした	書きかけなどで組めませんでした
ライブ表示を止めました	最後に組んだ PDF を表示したままです

組めなかったときは、ツールバーの下に理由を出し、前に組めた PDF を表示し続けます。詳しい内容は問題パネルで確認できます。原稿を直すと組み直します。

7.5.1 編集中の箇所を追う

カーソルを動かすと、編集中の箇所が見えるようにプレビューをスクロールし、その行に目印を出します。目印は、ページの左の縁の線と、行の薄い帯です。消えずにカーソルについて動きます。

- 目印が画面の中ほどにあるうちは、打つたびに揺れないようにスクロールしません。画面の外や端に近づくと、画面の上から 3 割のあたりに来るようにスクロールします。
- 組み直して PDF が入れ替わっても、表示している位置を保ちます。プレビューを読んでいる(エディタにフォーカスがない)ときは、組み直してもカーソルの位置へ引き戻さず、目印だけを動かします。
- 組んだ後に書き足した文は、まだ PDF にありません。書き足した場所(前の段落の終わりのあたり)に目印を出します。
- プレビューのページをダブルクリックすると、原稿のその箇所の行へ移ります。合本では章の md を開きます。
- 追わないようにするには、設定`pubpdf.liveFollow`をオフにします(設定パネルの最後でも変えられます)。ダブルクリックで原稿へ移る操作は、オフでも使えます。

ライブプレビューの PDF には、段落とリストの項目の先頭の位置を添付ファイル(`pubpdf-sync.json`)として書き込み、編集中の行を探すのに使います。見た目は変わりません。`Pubpdf: PDF を作成`で作る PDF には付けません。次のときは、位置がおおよそになります。

- Mermaid の図の中の行は、図のあたりに目印を出します。図は画像なので、行ごとの位置がありません。
- `pdf-standard`に PDF/A-2b(`a-2b`)を指定した文書は、添付ファイルを付けられないので、位置を書き込みません。見出しと PDF の文字から探すので、文字が PDF にない行(図や数式の中など)は、前後の見出しから見当を付けた位置になります。
- `include`で読み込んだコードの中をダブルクリックすると、その文字が原稿にないので、近くの行に移ります。

7.6 文章の点検

Markdown の原稿を書いている間に文章を点検し、問題パネルに出します。電球のアイコン(クイックフィックス)から 1 か所ずつ直すか、同じ種類の誤りを文書の中でまとめて直せます。`Pubpdf: 文章の誤りをまとめて直す`は、自動で直せるものを全部直します。コード、数式、URL、リンク先、HTML のタグ、frontmatter は点検しません。

表 7-4: 文章の点検

点検	設定(既定)	内容
和文と英数字の間の空白	`spacing` (`off`)	`no-space` は「日本語 eng 日本語」の空白を知らせて消します。 `space` は和文と英字の間に空白がないところを知らせます
文字の幅	`width` (オン)	全角の英数字(ｋ、Ａ、１)を半角に、半角のカナ(ｶﾅ)を全角にします
つづり	`spelling` (オン)	よくある英単語のつづりの誤り(recieve、seperate など)と、同じ語の重なり(the the)。語を足すときは VS Code の設定 `pubpdf.lint.words` か frontmatter の `words` に `{ 誤り: 正しい語 }` の形で書きます
文体	`style` (`off`)	`dearu` (である調。ビジネスや技術の文書)は「です」「ます」の文を、 `desumasu` (です・ます調)は「である」「だ」や普通の形(「する。」)の文を知らせます。 `consistent` は混在を知らせ、多い方にそろえます。段落の文の終わり(「。」の前)だけを見て、リスト、表、見出し、引用は見ません
日本語の誤り	`grammar` (オン)	ら抜き言葉(見れる → 見られる)、い抜き言葉(してる → している)、助詞の重なり(をを)、句読点の重なり(、。)
ひらがなで書く語	`notation` (オフ)	下さい → ください、出来る → できる、致します → いたします、この様に → このように、殆ど → ほとんど など

設定は VS Code の `pubpdf.lint.*` (`pubpdf.lint.style` など)で決め、文書ごとには frontmatter の `pubpdf.lint` に書きます。frontmatter が優先されます。 `lint: false` でその文書の点検を止め、 `pubpdf.lint.enabled` をオフにするとすべての文書で止めます。

```yaml
pubpdf:
  lint:
    style: dearu        # である調の文書
    spacing: no-space   # 和文と英数字の間に空白を入れない
    notation: true
```

- 文体の自動の修正は、形の決まった終わり方(「します」→「する」、「です」→「である」、「されます」→「される」など)だけです。「示します」(示す)のように見分けられないものは、知らせるだけにします。
- 英単語のつづりは、よくある誤りの一覧と照らし合わせるだけです。すべての誤りは見つけられません。

7.7 表の見やすい表示(試験的)

設定 `pubpdf.tableView.enabled` をオンにする(コマンド `Pubpdf: 表の見やすい表示をオン/オフ(試験的)` と、設定パネルの最後の「編集中の表示」でも切り替えられます)と、編集中のパイプ表を見やすくします。原稿の文字は変えません。

- 表の行を 1 行おきに塗り、行の終わりに ⏎ の印を付けます。長い行が折り返しても、どこまでが 1 行かが分かります。
- 見出しの行を太字にします。
- カーソルのない行は、列を揃えて表示します。原稿の空白で揃えていなくても、縦線が揃って見えます(原稿の空白は変えません)。全角の字は全角の空白で埋めるので、フォントによらず揃います。
- 長いセル(既定は半角 24 字。設定 `pubpdf.tableView.cellWidth`)は、後ろをたたんで「…」にします。たたんだセルにマウスを重ねると全部の文字を出します。
- カーソルを置いた行は、書いたとおりに出します。行を離れると揃います。
- 揃える表示とたたむ表示をやめるには `pubpdf.tableView.fold` をオフにします。VS Code の文字の飾りの仕組みで文字を見えなくしているので、VS Code の版によって崩れたときもオフにしてください。

表の行の終わりで Enter を押すと、次の行(空のセル)を足し、カーソルを最初のセルに置きます。見出しの行の終わりなら、区切りの行の次に足します。縦線を空白で揃えた表では、足す行も同じ幅にします。空の最後の行で Enter を押すと、その行を消して表を抜けます(空行を挟んで次の文を書けます)。取り消すと、改行ごと戻ります。この動きは表の見やすい表示がオフでも効きます。止めるには `pubpdf.tableView.enterNewRow` をオフにします。

7.8 エラーと警告

- エラーと警告を問題パネルに出します。章ファイル、画像、リンク、frontmatter の誤りは、選ぶと原稿の該当行へ移動します。
- 組版中のエラーは原稿の行に対応しないので、親の md の先頭に出します。内容は出力パネルの Pubpdf で確認できます。

7.9 設定のキー

VS Code の設定は、frontmatter に書いていない項目の既定値になります。 `font-paths` だけは例外で、frontmatter と VS Code の設定の両方を読み込みます。

表 7-5: VS Code の設定と frontmatter の対応

VS Code の設定	frontmatter のキー
`pubpdf.cover`	`cover`
`pubpdf.toc`	`toc` (`none` 、 `1` 、 `2` 、 `3`)
`pubpdf.listOfFigures`	`list-of-figures` (`on` 、 `off`)
`pubpdf.listOfTables`	`list-of-tables` (`on` 、 `off`)
`pubpdf.citationStyle`	`citation-style`
`pubpdf.footnotes`	`footnotes`
`pubpdf.footnotesTitle`	`footnotes-title`
`pubpdf.confidential.text`	`confidential`
`pubpdf.confidential.position`	`confidential-position`
`pubpdf.watermark`	`watermark`

VS Code の設定	frontmatter のキー
`pubpdf.header.left`、`pubpdf.header.center`、`pubpdf.header.right`	`header-left`、`header-center`、`header-right`
`pubpdf.footer.left`、`pubpdf.footer.center`、`pubpdf.footer.right`	`footer-left`、`footer-center`、`footer-right`
`pubpdf.pageNumber`	`page-number`（`auto` は既定に任せます）
`pubpdf.pageCount`	`page-count`
`pubpdf.paper`	`paper`
`pubpdf.orientation`	`orientation`
`pubpdf.columns`	`columns`
`pubpdf.margin.top`、`pubpdf.margin.bottom`	`margin.top`、`margin.bottom`
`pubpdf.margin.inner`、`pubpdf.margin.outer`	`margin.inner`、`margin.outer`
`pubpdf.numbering`	`numbering`
`pubpdf.figureNumbering`	`figure-numbering`（`auto` は既定に任せます）
`pubpdf.equationNumbering`	`equation-numbering`
`pubpdf.codeLineNumbers`	`code-line-numbers`（`on`、`off`）
`pubpdf.codeDisplay`	`code-display`
`pubpdf.codeFitMin`	`code-fit-min`
`pubpdf.codeAttach`	`code-attach`
`pubpdf.codeAttachNote`	`code-attach-note`
`pubpdf.codeAppendixTitle`	`code-appendix-title`
`pubpdf.codeAppendixNote`	`code-appendix-note`
`pubpdf.fontSize`	`font-size`
`pubpdf.lineSpacing`	`line-spacing`
`pubpdf.firstLineIndent`	`first-line-indent`（`on`、`off`）
`pubpdf.chapterBreak`	`chapter-break`（`on`、`off`）
`pubpdf.oddStart`	`book.odd-start`（`auto`、`on`、`off`）
`pubpdf.gutter`	`book.gutter`
`pubpdf.accentColor`	`accent-color`
`pubpdf.tableStyle`	`table-style`
`pubpdf.codeTheme`	`code-theme`
`pubpdf.inlineCodeBackground`	`inline-code-background`（`on`、`off`）
`pubpdf.tableFit`	`table-fit`（`on`、`off`）
`pubpdf.pdfStandard`	`pdf-standard`
`pubpdf.font.body`	`font`
`pubpdf.font.heading`	`font-heading`
`pubpdf.font.code`	`font-code`

VS Code の設定	frontmatter のキー
`pubpdf.font.math`	`font-math`
`pubpdf.font.paths`	`font-paths` (相対パスはワークスペースのフォルダが基準です)

次の設定は VS Code だけのものです。

表 7-6: VS Code だけの設定

設定	内容
`pubpdf.quartoPath`	Quarto の実行ファイルのパス
`pubpdf.outputDirectory`	PDF の出力フォルダ。frontmatter の `output` が優先されます
`pubpdf.buildDirectory`	中間ファイル(`.pubpdf`)の置き場所。`workspace` (既定)はワークスペースのフォルダの直下、 `document` は原稿と同じフォルダ、`global` は拡張機能の保管場所(プロジェクトのフォルダには作りません)
`pubpdf.renderOnSave`	保存時に PDF を作り直す
`pubpdf.liveDelay`	ライブプレビューで、編集を止めてから組み直すまでの時間(ミリ秒。既定は 1000)
`pubpdf.liveFollow`	ライブプレビューで、編集中の箇所が見えるようにプレビューをスクロールし、その行に目印を出す(既定はオン)
`pubpdf.diagramCache`	前に描いた Mermaid の図を再利用する(既定はオン)
`pubpdf.lint.enabled`、 `pubpdf.lint.*`	文章の点検。frontmatter の `pubpdf.lint` が優先されます
`pubpdf.tableView.enabled`、`pubpdf.tableView.fold`、`pubpdf.tableView.cellWidth`	表の見やすい表示(試験的)
`pubpdf.tableView.enterNewRow`	表の行の終わりの Enter で次の行を足す(既定はオン)

ライブプレビューと表の見やすい表示の設定は、設定パネルの最後の「編集中の表示」でも変えられます。frontmatter には書けません。

第 8 章　困ったときは

まず環境を確認してください。コマンドパレットで `Pubpdf: 環境を確認` を実行します。

エラーと警告は、VS Code では問題パネルに出ます。原稿の行が分かるものは、選ぶとその行へ移動します。Quarto の生のログは、出力パネルの Pubpdf で確認できます。

8.1　エラーで止まるとき

表 8-1: エラーの原因と対処

表示	原因	対処
`... が見つかりません` (原稿のパス)	コマンドに渡したパスの誤り	実行場所からの相対パスを確認します
`章ファイル ... が見つかりません`	`chapters` のパスの誤り	親 md からの相対パスを確認します
`章ファイル ... は md ではありません`	md 以外のファイルを指している	拡張子が `.md`、`.markdown`、`.qmd` のファイルを指定します
`... は URL かネットワーク上の場所なので、読み込みません`	章ファイル、ロゴ、文献のファイルに URL か `//サーバ/共有` の場所を指定した	ファイルを原稿のフォルダに置いて、相対パスで指定します
`本文に YAML と見なされる部分があるので、安全のため組みません`	本文の `---` の行を、Quarto が設定と読んでしまう形になっている	`---` の行を `***` に変えるか、消します
`原稿と画像が別のドライブにある`	章や画像が別のドライブを指している	同じドライブに置きます
`画像 ... が見つかりません`	画像のパスの誤り	表示された行のパスを確認します。`画像 a.png =200x が見つかりません` のように幅まで含めて表示されるときは、Pubpdf が 1.6.0 より前の版で、幅の書き方(`=200x`)を読めません。`code --list-extensions --show-versions` で版を確かめ、更新してください
`参考文献のファイル ... が見つかりません`	`bibliography` のパスの誤り	原稿からの相対パスを確認します
`ロゴのファイル ... が見つかりません`	`logo` のパスの誤り	原稿からの相対パスを確認します
`ERROR: [object ErrorEvent]`	Mermaid の図を描くブラウザを起動できなかった	Mermaid を含む文書を同時に組んでいるときは、1 つずつ組みます
`embedding PDFs is currently not supported`	PDF の規格を指定した文書に、PDF の画像がある	PNG か SVG にします
`failed to parse BibLaTeX`	`.bib` の書き方の誤り	括弧の閉じ忘れを確認します。エントリは波括弧で書きます

表示	原因	対処
`... は画像として使えません`	画像以外のファイルを指している	png、jpg、gif、svg、webp、pdf を指定します
`frontmatter の YAML を読めません`	YAML の書き方の誤り	引用符の閉じ忘れや字下げを確認します
`... を上書きできません`	PDF を他のアプリで開いている	PDF を閉じて、もう一度実行します
`Quarto が見つかりません`	未導入か、場所が不明	導入するか、`pubpdf.quartoPath` (VS Code)か `PUBPDF_QUARTO` で指定します
`レンダリングに失敗しました`	組版中のエラー	続く `error:` の内容を確認します

組版中のエラーが指す行番号は、原稿ではなく生成した Typst のファイルの行です。組版エラーで止まったときは、そのファイルが中間フォルダの `<原稿名>.typ` (合本では `index.typ`)に残っています。中間フォルダは、既定ではワークスペースのフォルダの直下の `.pubpdf/build/<原稿のフォルダ>/<原稿名>/` です(VS Code の設定 `pubpdf.buildDirectory` で変わります)。

8.2 警告が出るとき

警告が出ても PDF は作られます。ただし、意図どおりの結果にならないことがあるので、内容を確認してください。

表 8-2: 警告の意味

表示	意味
`未知のキー ... は無視します`	キーの書き間違いの可能性があります
`... の値 ... は使えません`	既定値で組みました
`frontmatter のキー ... は使いません`	Quarto の設定は書けません
`本文の途中の YAML は読みません`	frontmatter は原稿の先頭に書いてください
`リンク先 ... はこの文書に含まれない`	リンクを文字だけにしました
`リンク先の見出し ... が見つからない`	リンクを文字だけにしました
`章の pubpdf: は無視します`	親 md に書いてください
`title と H1 の両方があります`	H1 を章題にしました
`章題がありません`	ファイル名を章題にしました
`UTF-8 で読めない文字があります`	原稿を UTF-8 で保存し直してください
`日本語フォントが見つかりません`	フォントを導入するか、`fonts/` に置いてください
`Quarto ... は古い版です`	Quarto を更新してください
`Unable to resolve crossref`	`@fig-` などの参照先がありません
`フォント ... が見つかりません`	フォント名か読み込み先を確認してください。別のフォントで組みました
`フォントの場所 ... が見つかりません`	`font-paths` のパスを確認してください

表示	意味
`current font is not designed for math`	`font-math` には数式用のフォントを指定してください
数式を変換できない	使えない LaTeX の命令があります。数式が LaTeX のまま出ています
`:::` の囲みを閉じる行がありません	囲みの最後に `:::` の行を足してください
引用やリストの中では改ページできない	引用、囲み、リストの外に書いてください。改ページは無視しました
引用、囲み、リストの中では横向きにできない	`.landscape` の囲みを外に出してください。縦のまま組みました
参考文献のファイルに `...` がない	`.bib` のキーを確認してください。文字のまま出しました
`[@...]` を文字のまま出します	文献を引用するときは `bibliography` を指定してください
`{{< meta ... >}}` は使えません	使えるキーは `title`、 `version` など 7 つです
`...` に差し込む値が `frontmatter` にありません	frontmatter に値を書いてください。空にしました
`abstract` は文字で書いてください	リストなどは使えません。概要は出しませんでした
余白か綴じ代が大きすぎて本文が入らない	`margin` や `book.gutter` の値を小さくしてください。既定の値で組みました
`...` の図を読めません	`tree` や `chart` などの図の書き方の誤りです。コードブロックのまま出しました。示された行を直してください
図の `label` に `...` は使えません	`label` は `fig-` で始めてください
段の幅が `30mm` より狭くなる	段組みにせず 1 段で組みました。 `columns` を減らすか余白を小さくしてください
段組みの `count` に `...` は使えない	2 段で組みました。 `count` は 2 から 4 までです
`{.column}` と他の内容が混ざっている	幅の指定を使わず、流れる段組みにしました
表紙がないので `logo` は使いません	`cover` に `simple` か `formal` を指定してください
画像 `...` に説明がない、 `Mermaid` の図に題がない	PDF/UA の代替テキストを「図」にしました。説明か題(`%%| fig-cap:`)を書いてください
画像 `...` は原稿のフォルダの外にあります	自分の原稿なら問題ありません。受け取った原稿では、意図しない画像でないかを確かめてください
画像 `...` は URL かネットワーク上の場所なので、読み込みません	画像の場所に灰色の枠を入れました。原稿のフォルダに保存して、相対パスで書いてください
見出しに番号がないので、図表は通し番号にします	`figure-numbering` の `chapter` と `section` には `numbering` の指定が必要です
見出しに番号がないので、`heading-format` は使いません	`numbering` に `chapter` か `decimal` を指定してください

表示	意味
`Mermaid の図が大きいので、文字が約 ...pt まで縮みます`	図が版面の幅かページの高さに収まらず、文字が 6pt より小さくなります。図を分けるか、向きを変えるか、横向きのページに置いてください
`グリッド表の縦線の位置が罫線と合いません`	示された行の `|` の位置を、罫線の `+` に揃えてください。全角の文字は 2 文字分です
`HTML の表を閉じる </table> がない`	表の最後に `</table>` を書いてください。表にせずに出しました
`HTML の表の中の画像は出しません`	画像は Markdown の記法で、表の外に書いてください
`前に空行がないので、見出しになりません`	示された行の前に空行を入れてください。リスト、表、引用も同じです。前の行の文字としてつなげて組みました
`画像の高さだけの指定(=x...)には対応しない`	幅で `=200x` のように書いてください。元の大きさで出しました
`画像の高さの指定(x...)は使いません`	幅だけを使いました
`図の幅 ... は使えない`	`%%| width:` は `200` (px)か `50%` のように書いてください。指定を無視しました
`図の寄せ ... は使えない`	`%%| fig-align:` は `left`、 `center`、 `right` のどれかにしてください。指定を無視しました
`approval は role(役割)を持つ項目のリストで…`	`approval` の各項目に `role` を書いてください。承認欄は出しませんでした
`脚注 [^...] の本文([^...]: ...)が見つかりません`	`[^1,2]` のように並べた脚注のうち、本文を書いていないものがあります。その番号は文字のまま出ました
`表のセルの中の入れ子のリストは書けません`	パイプ表のセルの `<ul>` の中の `<ul>` は出せません。入れ子にするときはグリッド表で書いてください

8.3 仕上がりがおかしいとき

表 8-3: 仕上がりの問題と対処

症状	原因	対処
文字全体が細く、太字が効かない	Quarto 1.9 系と可変フォントの組み合わせ	Quarto を更新するか、`font` で BIZ UDPGothic などを指定します
意図しない白紙	奇数ページ始まりがオン	`book.odd-start: false` にします。合本では既定でオンです
数式の記号が崩れる	数式用でないフォントを `font-math` に指定した	数式用のフォントを指定するか、`font-math` を消します
プレビューが古いまま	PDF を作り直していない	作成のボタンを押します。作り直すと表示位置を保ったまま更新します
ライブプレビューが原稿に追いつかない	組み直している途中か、書きかけで組めない	ツールバーの表示を見ます。「組み直しています…」なら待ちます。「組めませんでした」なら、ツールバーの下の理由と問題パネルを見て直します(ライブプレビュー)
本文に `?@fig-名前` と出る	参照先の図がない	図の `{#fig-名前}` を確認します
Mermaid の図でエラーになる	ブラウザが見つからない	`Pubpdf: 環境を確認` でブラウザの有無を確認します
PDF をメールで送れない、相手が受け取れない	コードを PDF の添付ファイルにしている(1.6 までの既定)	`code-attach: none` (1.7 からの既定)にし、`.attach` と `code-display: attach` を使わないようにします。1 ページに入らないコードは付録へ移ります
縦に長い図の題がページの下にはみ出す	画像がページの高さまで大きい	画像に `.fit` を書きます(原稿の書き方)
図に `.fit` を書いても変わらない	1.6 までは図の `.fit` がない(表とコードだけ)	1.7 に更新します

8.4 既知の制限

- 番号付きの callout(`::: {#nte-名前 .callout-note}`)は、題の欄が少し縦に広くなります。
- VS Code のプレビューでは、文字の選択、リンクの移動はできません。必要なときは「他のアプリで開く」を使います。
- `aligned` などの複数行の数式で `\tag` を複数使うと、最後の 1 つだけが出ます。
- 引用、囲み、リストの中の見出しは、目次に載りません。
- 余白の注(`.column-margin`)には、まだ対応していません。本文の領域に出ます。
- 段の長さを揃えられないことがあります。図や表などを含む段組み、ページをまたぐ段組みの最後のページ、文書全体の段組みの最後のページです。
- 動作を確かめたのは Windows 11 だけです。

8.5 動作を確かめた Quarto の版

機能確認用の文書を各版で組み、結果を比べました。

表 8-4: 確認した Quarto の版

版	単一の md	合本	結果
1.11.9	動く	動く	1.10.18 と同じ仕上がり
1.10.20	動く	動く	1.10.18 と同じ仕上がり
1.10.18	動く	動く	基準にした版
1.9.36	動く	動く	Noto Sans JP が細字になり、太字も効かない
1.8.27	動かない	動かない	組版エラーで止まる